COMP229 — Web Application Development

Mid-Term Test

Fran's Campus Lost & Found REST API

REST API Test Evidence — Postman & MongoDB Compass

Francis Torres

Student ID: 301530008

Project: COMP229-MidTerm-301530008

https://github.com/francismtorres/COMP229-MidTerm-301530008

October 23, 2026

Contents

Overview & Environment 3

Server start-up log (connected to MongoDB, listening on 8080) 3

How the tests are organised 3

Status codes implemented 3

Test Results Summary 4

Evidence: updating or deleting another user's item is rejected 6

AI Usage Disclosure 7

What the AI did, mapped to the permitted uses 7

Uses not explicitly on the permitted list 7

What was not AI-generated 8

My responsibility 8

Screenshots 9

1. Server running on port 8080 9

2. Authentication (provided signup / signin / JWT) 9

3. Create items — POST /api/items (Bearer) 13

MongoDB evidence — Create items 17

4. Read items — GET (public and Bearer) 19

5. Update items — PUT /api/items/:id (Bearer, owner only) 25

MongoDB evidence — Update items 29

6. Delete items — DELETE (Bearer, owner only) 30

MongoDB evidence — Delete items 34

Overview & Environment

This document records the testing of Fran's Campus Lost & Found REST API, built on the provided starter project (Node.js, Express, MongoDB, Mongoose, TypeScript). Signup/signin and the JWT middleware were provided; the Item model validation, HTTP input validation, the seven item controllers and the item routes were implemented for this test.

Every request was sent from Postman against the running server, and the resulting data changes were verified in MongoDB Compass. Two users were created so that the owner-only rules could be tested: Francis Torres (owner) and Alex Other (a second user who tries to change Fran's items).

Item	Value
Node.js	v24.21.0
Framework	Express 4 + TypeScript (ESM), Mongoose 8
Database	MongoDB 8.3 Community, local (mongodb://127.0.0.1:27017/comp229_midterm_fran)
Port	8080 (PORT=8080 in .env)
Run command	npm install → npm run build → npm start (or npm run dev)
REST client	Postman 12 — collection: postman/Fran-LostFound.postman_collection.json
DB viewer	MongoDB Compass 1.53
Source code	https://github.com/francismtorres/COMP229-MidTerm-301530008

Server start-up log (connected to MongoDB, listening on 8080)

```
> comp229-midterm-301530008@2.0.0 start
> node dist/server.js
[db] Connected to MongoDB: comp229_midterm_fran
[server] Listening on http://localhost:8080 (development)
```

How the tests are organised

The Postman collection is split into folders that are run top to bottom. Signin requests save the JWT into collection variables ({{franToken}}, {{alexToken}}), and create requests save the new item ids ({{franItemId}}, {{franItem2Id}}). Every request has Postman test scripts that assert the expected status code; the whole collection was also run with Newman (Postman's command-line runner): 44 requests, 56 assertions, 0 failures.

Status codes implemented

Situation	Status	Example (figure)
Successful GET / PUT / DELETE	200	Fig. 22, 33, 47
Successful create	201	Fig. 14
Invalid body, invalid filter, malformed id or JSON	400	Fig. 11, 26, 28, 13
Missing / invalid JWT	401	Fig. 9, 10
Not the item owner	403	Fig. 34, 43
Item not found	404	Fig. 29, 48
Duplicate title (unique field)	409	Fig. 16, 39

Test Results Summary

Every row below matches a screenshot later in this document. "Actual" is the status shown in the Postman response pane of that screenshot.

Fig.	Method	URL	Scenario	Status	Result
1	GET	/	API root confirms the server is up on port 8080 and personalised for Fran	200 OK	Pass
2	POST	/api/auth/signup	Sign up user #1 (Francis Torres) and receive a JWT	201 Created	Pass
3	POST	/api/auth/signup	Sign up again with the same email	409 Conflict	Pass
4	POST	/api/auth/signin	Sign in with a wrong password	401 Unauthorized	Pass
5	POST	/api/auth/signin	Sign in Fran; the test script stores the token in {{franToken}}	200 OK	Pass
6	POST	/api/auth/signup	Sign up user #2 (Alex Other), used to prove ownership rules	201 Created	Pass
7	POST	/api/auth/signin	Sign in Alex; token stored in {{alexToken}}	200 OK	Pass
8	GET	/api/auth/me	Bearer token resolves to the signed-in user	200 OK	Pass
9	POST	/api/items	Create without an Authorization header	401 Unauthorized	Pass
10	POST	/api/items	Create with a forged token ("Bearer not.a.real.token"; Postman masks the literal value as a vault variable)	401 Unauthorized	Pass
11	POST	/api/items	Only a title is sent; every missing required field is reported	400 Bad Request	Pass
12	POST	/api/items	Invalid type, category and a future date are rejected	400 Bad Request	Pass
13	POST	/api/items	Malformed JSON body	400 Bad Request	Pass
14	POST	/api/items	Fran creates "Black Jansport Backpack"; owner is set from the JWT and status defaults to Open	201 Created	Pass
15	POST	/api/items	Fran creates "White AirPods Pro Case"	201 Created	Pass
16	POST	/api/items	Duplicate title in different case ("black jansport backpack") is rejected; title is the unique field	409 Conflict	Pass
17	POST	/api/items	Alex creates "Web Application Development Textbook"	201 Created	Pass
22	GET	/api/items	List all items (public); test asserts 3 items	200 OK	Pass
23	GET	/api/items?type=Found	Filter by type; every result is Found	200 OK	Pass
24	GET	/api/items?category=Electronics	Filter by category	200 OK	Pass
25	GET	/api/items?type=Found&category=Books	Combined filters; exactly 1 match	200 OK	Pass
26	GET	/api/items?type=Stolen	Unknown filter value	400 Bad Request	Pass
27	GET	/api/items/{{franItemId}}	Get a single item by id (public, owner populated)	200 OK	Pass
28	GET	/api/items/not-a-valid-id	Malformed ObjectId	400 Bad Request	Pass
29	GET	/api/items/64b000000000000000000000	Valid ObjectId that does not exist	404 Not Found	Pass
30	GET	/api/items/mine	Fran's items only (Bearer {{franToken}})	200 OK	Pass
31	GET	/api/items/mine	Postman Test Results: "Only Fran's 2 items" PASSED (Alex's item is excluded)	200 OK	Pass
32	GET	/api/items/mine	Protected route without a token	401 Unauthorized	Pass

Fig.	Method	URL	Scenario	Status	Result
33	PUT	/api/items/{{franItemId}}	Owner (Fran) marks the backpack Returned and changes its location	200 OK	Pass
34	PUT	/api/items/{{franItemId}}	OWNERSHIP: Alex tries to rename Fran's item to "Hijacked by Alex" — rejected	403 Forbidden	Pass
35	PUT	/api/items/{{franItemId}}	Same request, Headers tab: it is sent with Alex's token (Bearer {{alexToken}})	403 Forbidden	Pass
36	PUT	/api/items/{{franItemId}}	Update without a token	401 Unauthorized	Pass
37	PUT	/api/items/{{franItemId}}	Invalid status and an attempt to overwrite owner	400 Bad Request	Pass
38	PUT	/api/items/{{franItemId}}	Empty body {}	400 Bad Request	Pass
39	PUT	/api/items/{{franItem2Id}}	Rename to a title already used by another item	409 Conflict	Pass
40	PUT	/api/items/12345	Malformed ObjectId	400 Bad Request	Pass
41	PUT	/api/items/64b000000000000000000000	Item does not exist	404 Not Found	Pass
43	DELETE	/api/items/{{franItemId}}	OWNERSHIP: Alex tries to delete Fran's item (Bearer {{alexToken}}) — rejected	403 Forbidden	Pass
44	DELETE	/api/items/{{franItemId}}	Delete without a token	401 Unauthorized	Pass
45	DELETE	/api/items/abc	Malformed ObjectId	400 Bad Request	Pass
46	DELETE	/api/items/64b000000000000000000000	Item does not exist	404 Not Found	Pass
47	DELETE	/api/items/{{franItemId}}	Owner (Fran) deletes the backpack	200 OK	Pass
48	GET	/api/items/{{franItemId}}	The deleted item is gone	404 Not Found	Pass
49	DELETE	/api/items/mine	Delete all of Fran's items; deletedCount = 1 (the AirPods)	200 OK	Pass
50	GET	/api/items/mine	Fran now has no items: []	200 OK	Pass
51	GET	/api/items	Alex's item is unaffected by Fran's DELETE /mine	200 OK	Pass

Evidence: updating or deleting another user's item is rejected

The items were created by **Francis Torres**. Signed in as **Alex Other**, the same requests were sent with Alex's JWT. The API compares item.owner with the user id taken from the verified token, and returns **403 Forbidden** before touching the document:

- **Fig. 34 & 35:** PUT by Alex → 403 "Forbidden: you can only update your own items".
- **Fig. 43:** DELETE by Alex → 403 "Forbidden: you can only delete your own items".
- **Fig. 42:** MongoDB still shows the original title, so the rejected update changed nothing.
- **Fig. 51 & 52:** Fran's DELETE /api/items/mine removed only Fran's items; Alex's item remains.

AI Usage Disclosure

This disclosure follows the course policy on permitted uses of AI. The same statement is in AI_DISCLOSURE.md in the repository.

Item	Detail
Tool	Claude Code (Anthropic), desktop app
Model	Claude Opus 5.5 (claude-opus-5-5)
When	October 23, 2026, one working session
Access given	The project folder, a terminal, and on-screen control of Postman and MongoDB Compass (each approved by me when requested)

What the AI did, mapped to the permitted uses

What the AI did	Permitted use it falls under
Read the starter project and specification, then proposed how to structure the work (validation middleware, controller responsibilities, status-code mapping).	Help create or revise implementation plans; explore possible architectural or implementation approaches
Wrote the Item model validation, validateItem.ts middleware, the seven item controllers, the route wiring and the error-handler additions from the specification.	Help translate a clearly defined specification into code
Chose a case-insensitive unique index on title and owner checks via item.owner.equals(req.userId); explained why set() + save() is used so update validation runs.	Compare alternative implementations or architectural choices; clarify APIs, frameworks, or library usage
Designed the test scenarios and edge cases (invalid ids, forged tokens, malformed JSON, duplicate titles in a different case, a second user modifying another user's item).	Generate possible test cases and edge cases
Wrote the Postman collection with test scripts and ran it with Newman (44 requests, 56 assertions, 0 failures).	Help formulate tests for APIs or services
Found and fixed problems during testing: an unclear malformed-JSON error message, and (in a final pre-submission review) "status": null being accepted on create. Re-ran type checking, the unit tests and the full Postman suite after each fix.	Help identify and explain programming errors; help interpret error messages and test failures
Reviewed the code for security issues and defects, e.g. rejecting a client-supplied owner and query-operator injection such as ?type[$ne]=x; ran a final check from a fresh clone of the repository on a clean database.	Review code for readability, maintainability, security, or possible defects
Wrote code comments, IMPLEMENTATION.md, the README updates and the commit message.	Assist with documentation and comments

Uses not explicitly on the permitted list

For full transparency, the AI also did the following. The list of permitted uses does not name these directly:

- Ran the Postman requests and took the screenshots. With my approval, the AI controlled Postman and MongoDB Compass on my screen, sent each request and captured the 52 screenshots in this document. The screenshots are real and unedited: every request went to the running API and every Compass view shows the actual database.
- Assembled this test-evidence document, including the captions and the results summary table.
- Handled project logistics: renaming the folder, starting the local MongoDB server, creating the git repository and ZIP archive, and pushing to GitHub after I created the empty repository.

What was not AI-generated

- The starter project (signup/signin, JWT middleware, User model, project configuration) was provided by the course.
- The test data is fictional; the two accounts are test accounts created only for this project.

My responsibility

I directed the work, approved each action the AI took on my computer, and reviewed the final code, tests and screenshots. I am responsible for this submission and can explain how each endpoint works, why each status code is returned, and how ownership is enforced.

Screenshots

1. Server running on port 8080

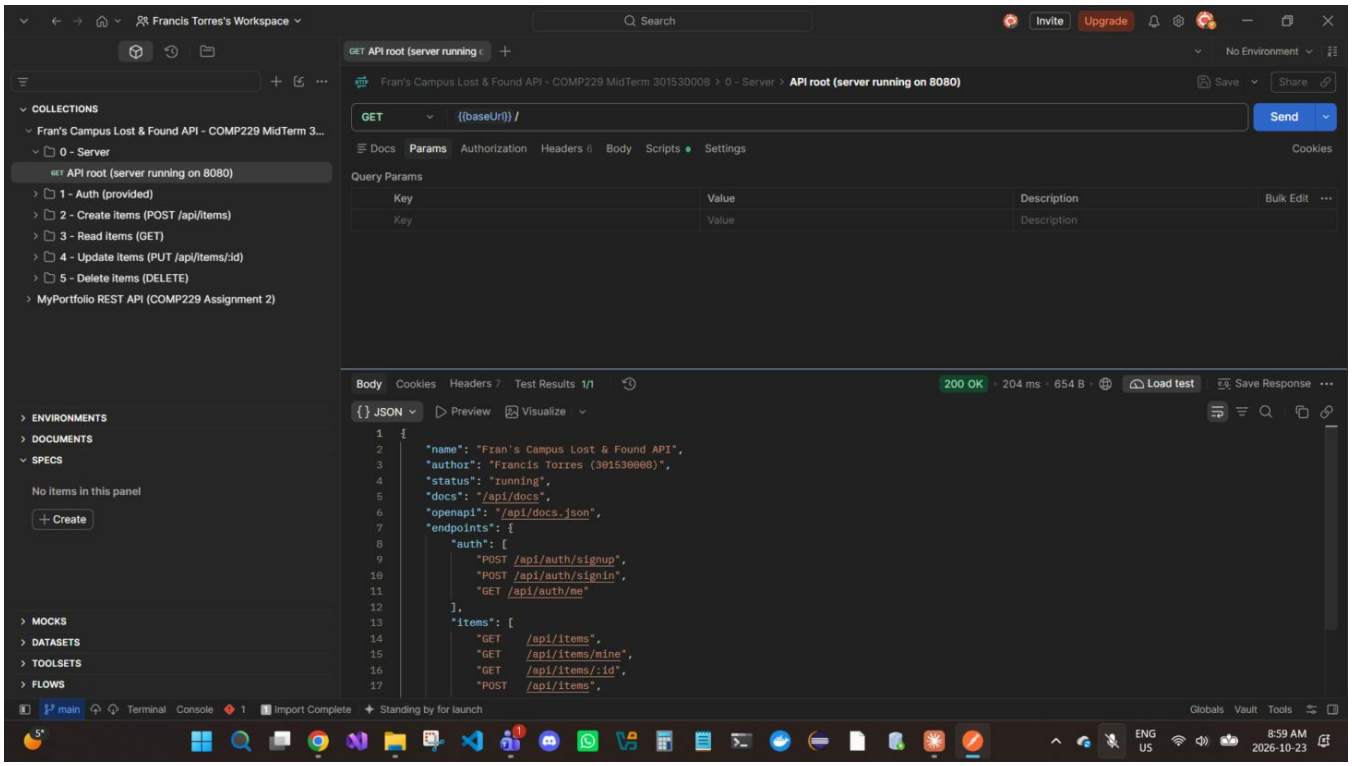


Figure 1 — GET / — API root confirms the server is up on port 8080 and personalised for Fran → **200 OK**

2. Authentication (provided signup / signin / JWT)

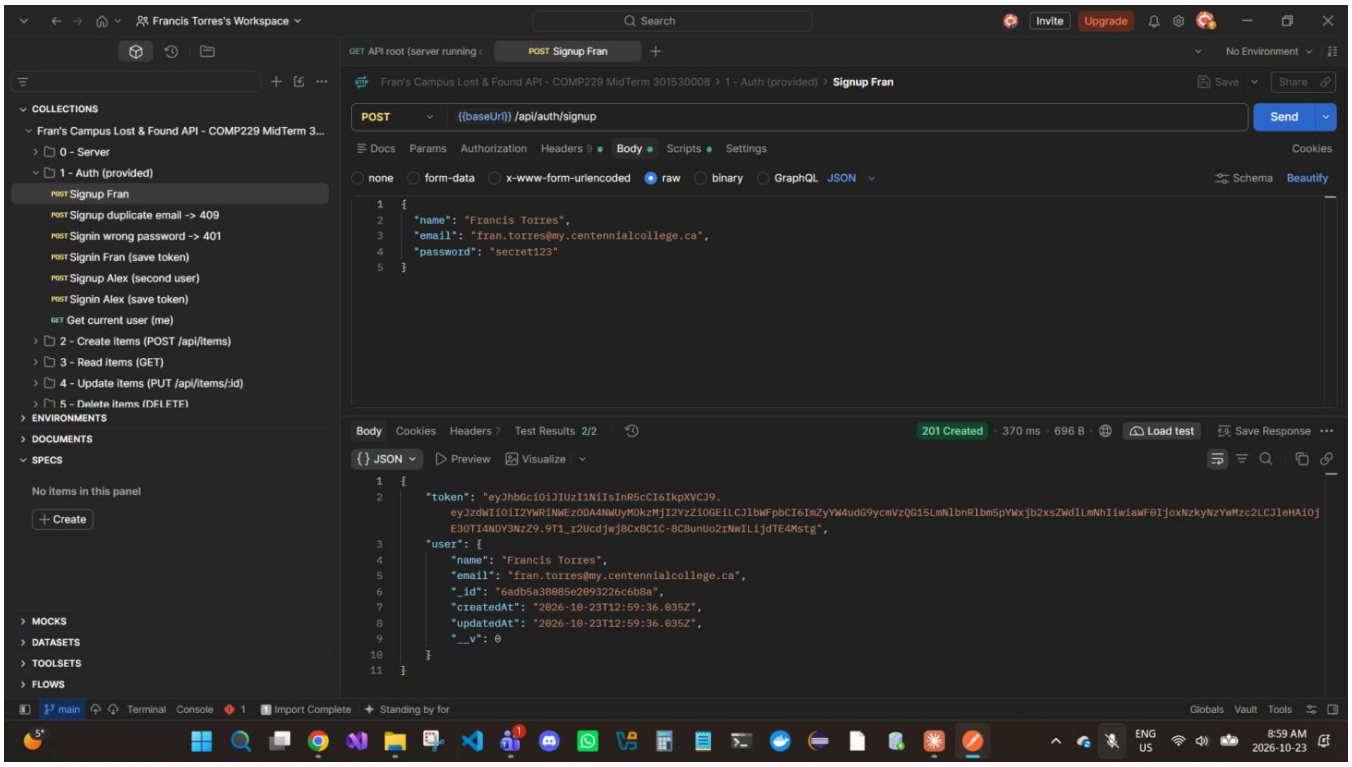


Figure 2 — POST `/api/auth/signup` — Sign up user #1 (Francis Torres) and receive a JWT → **201 Created**

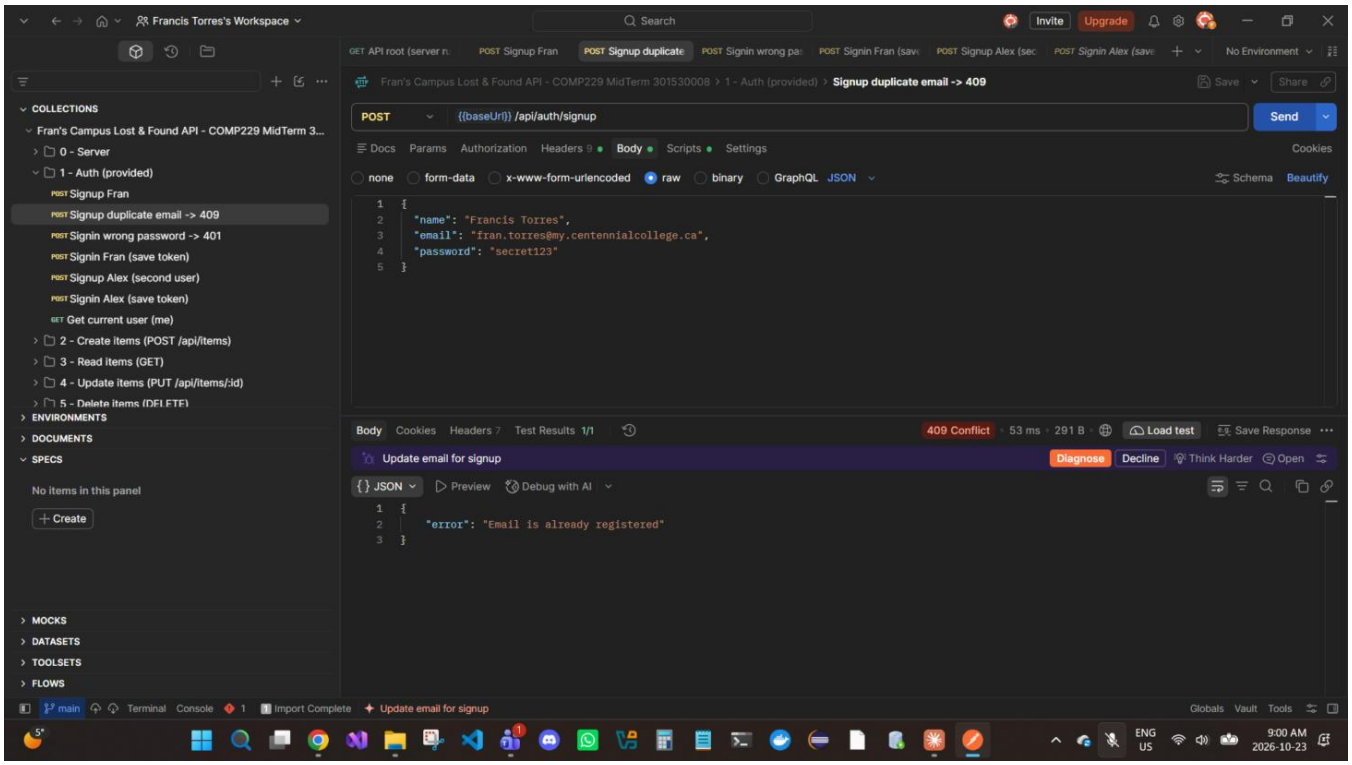


Figure 3 — POST `/api/auth/signup` — Sign up again with the same email → **409 Conflict**

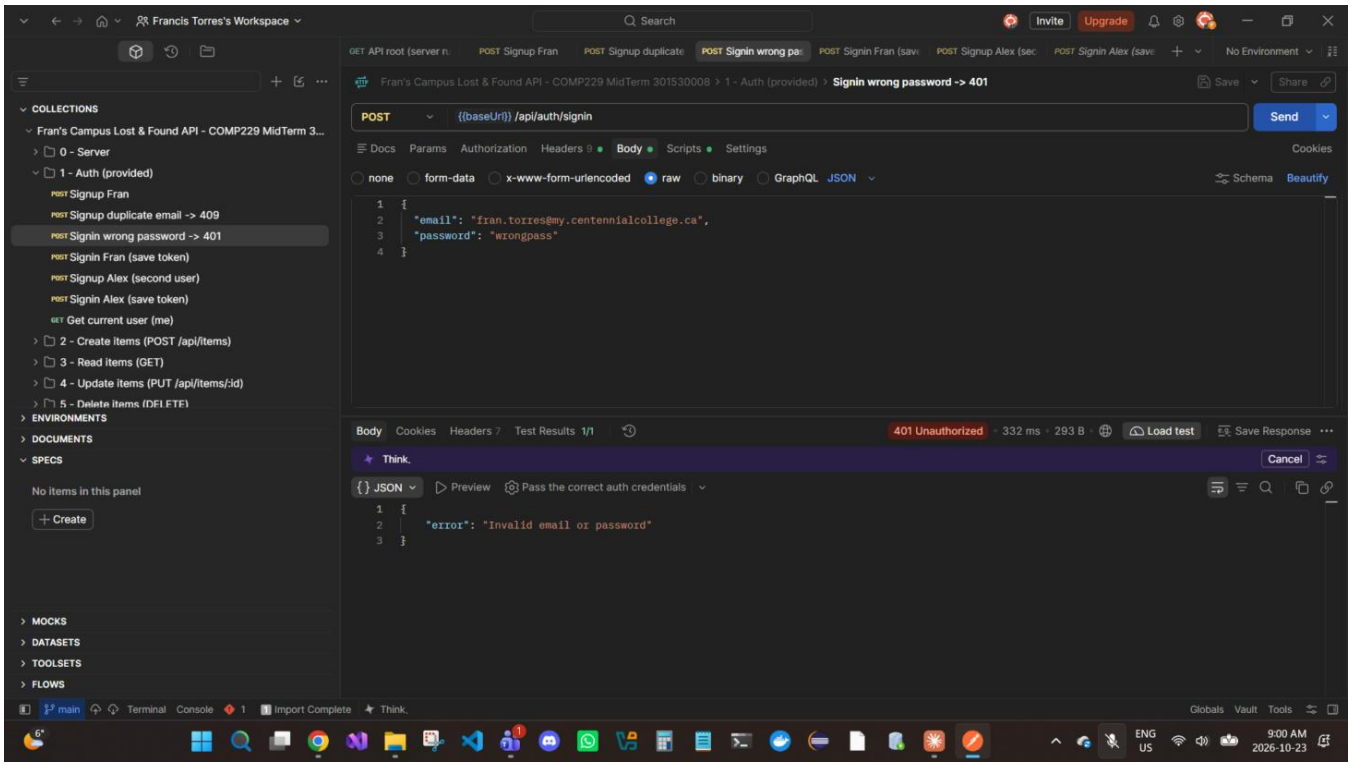


Figure 4 — POST `/api/auth/signin` — Sign in with a wrong password → **401 Unauthorized**

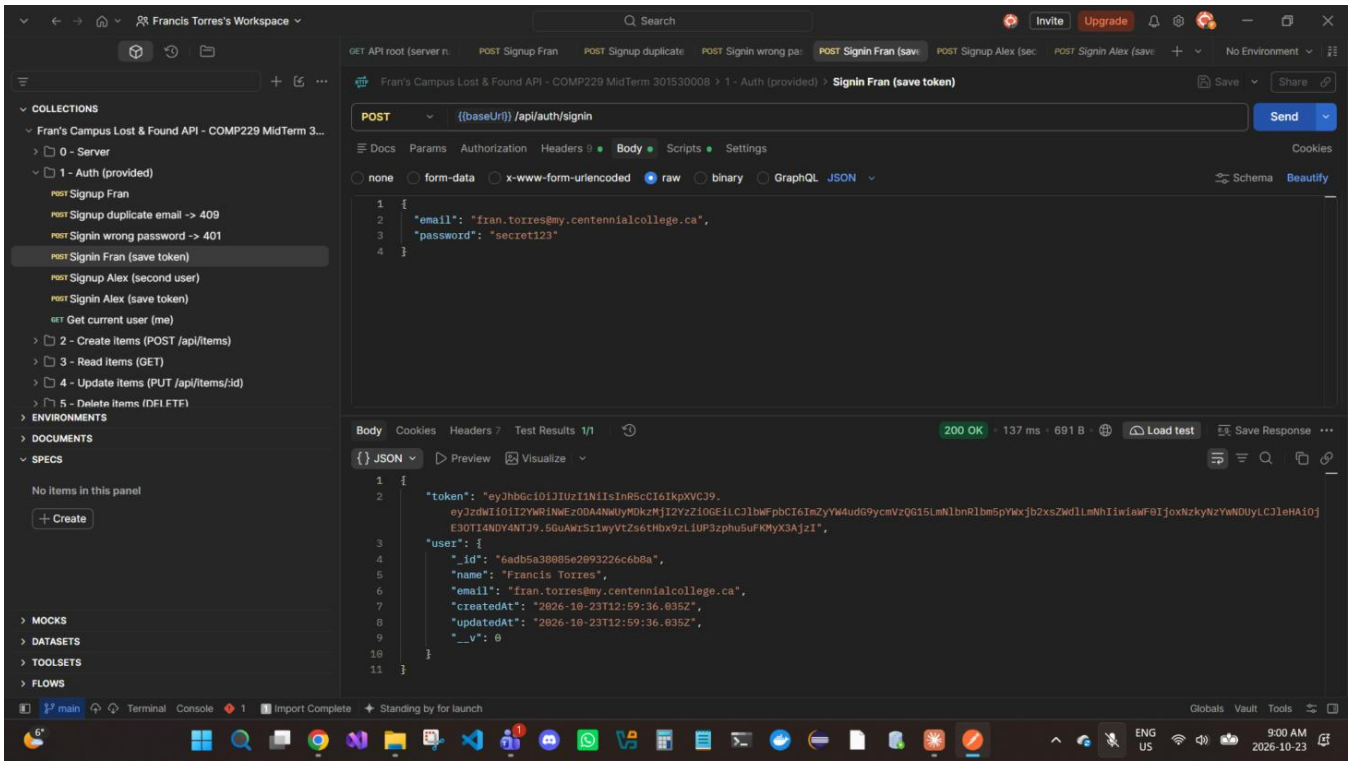


Figure 5 — `POST /api/auth/signin` — Sign in Fran; the test script stores the token in {{franToken}} → **200 OK**

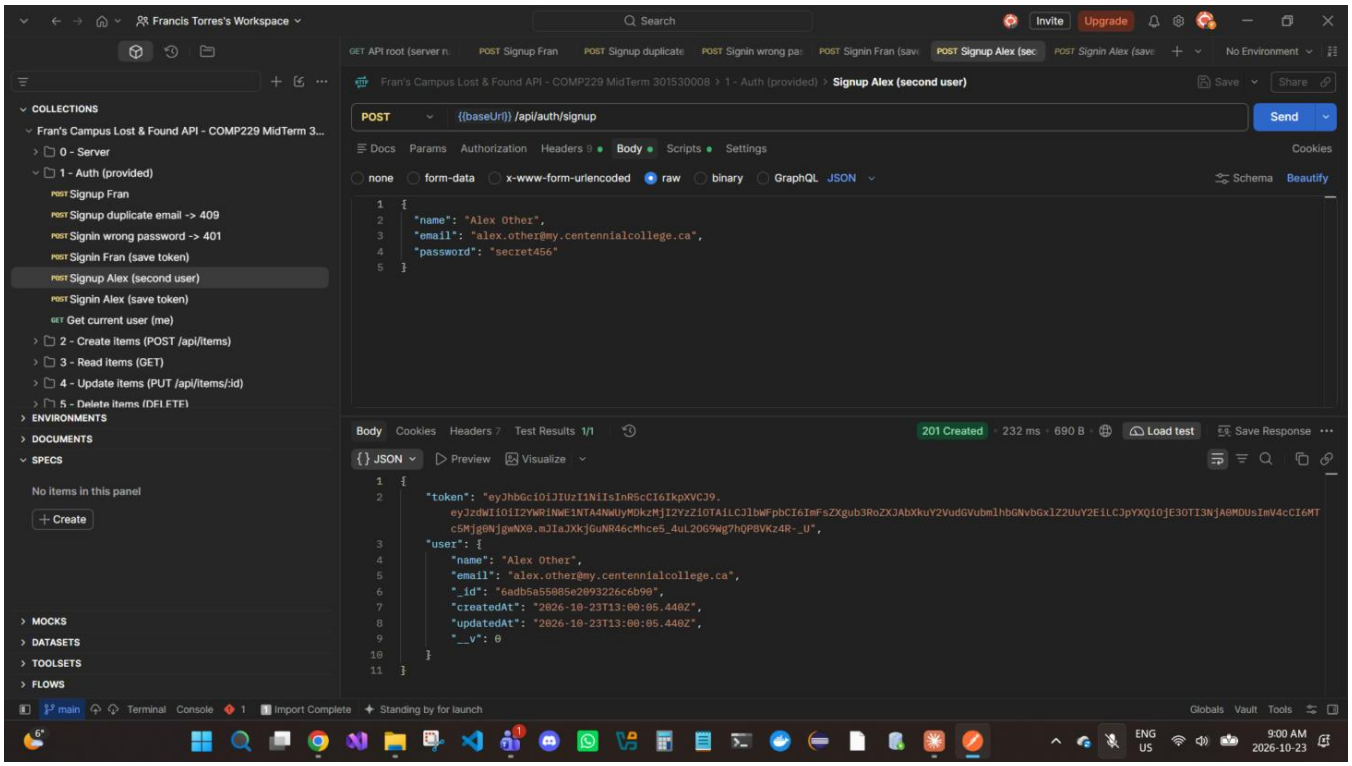


Figure 6 — `POST /api/auth/signup` — Sign up user #2 (Alex Other), used to prove ownership rules → **201 Created**

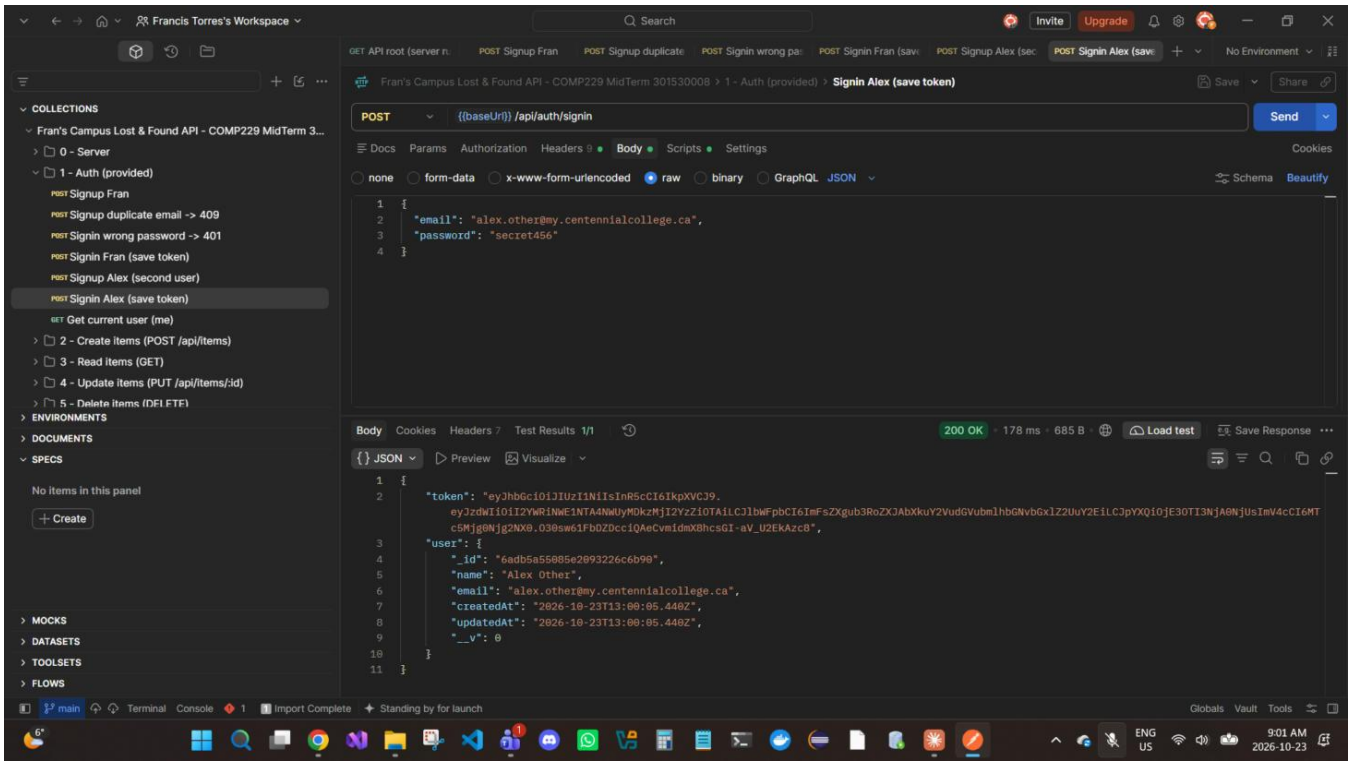


Figure 7 — `POST /api/auth/signin` — Sign in Alex; token stored in {{alexToken}} → **200 OK**

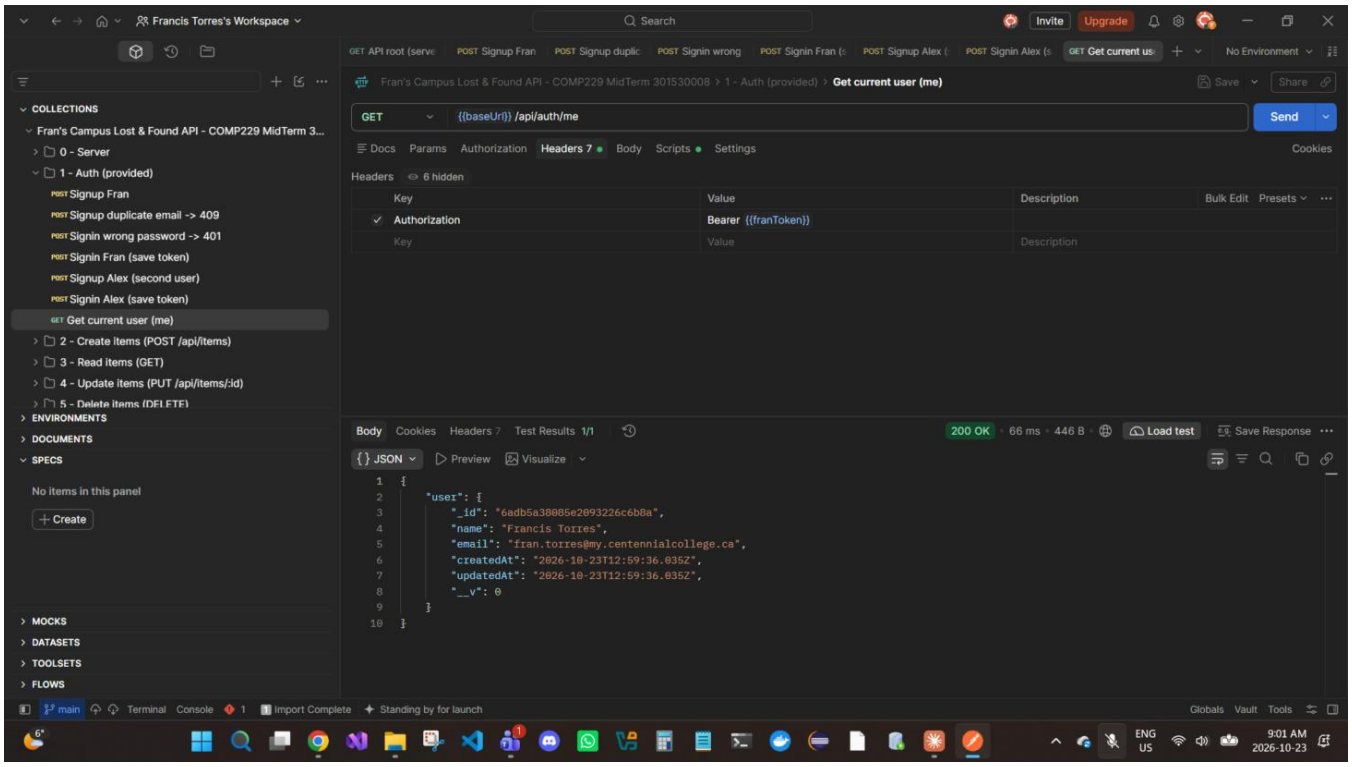


Figure 8 — `GET /api/auth/me` — Bearer token resolves to the signed-in user → **200 OK**

3. Create items — POST /api/items (Bearer)

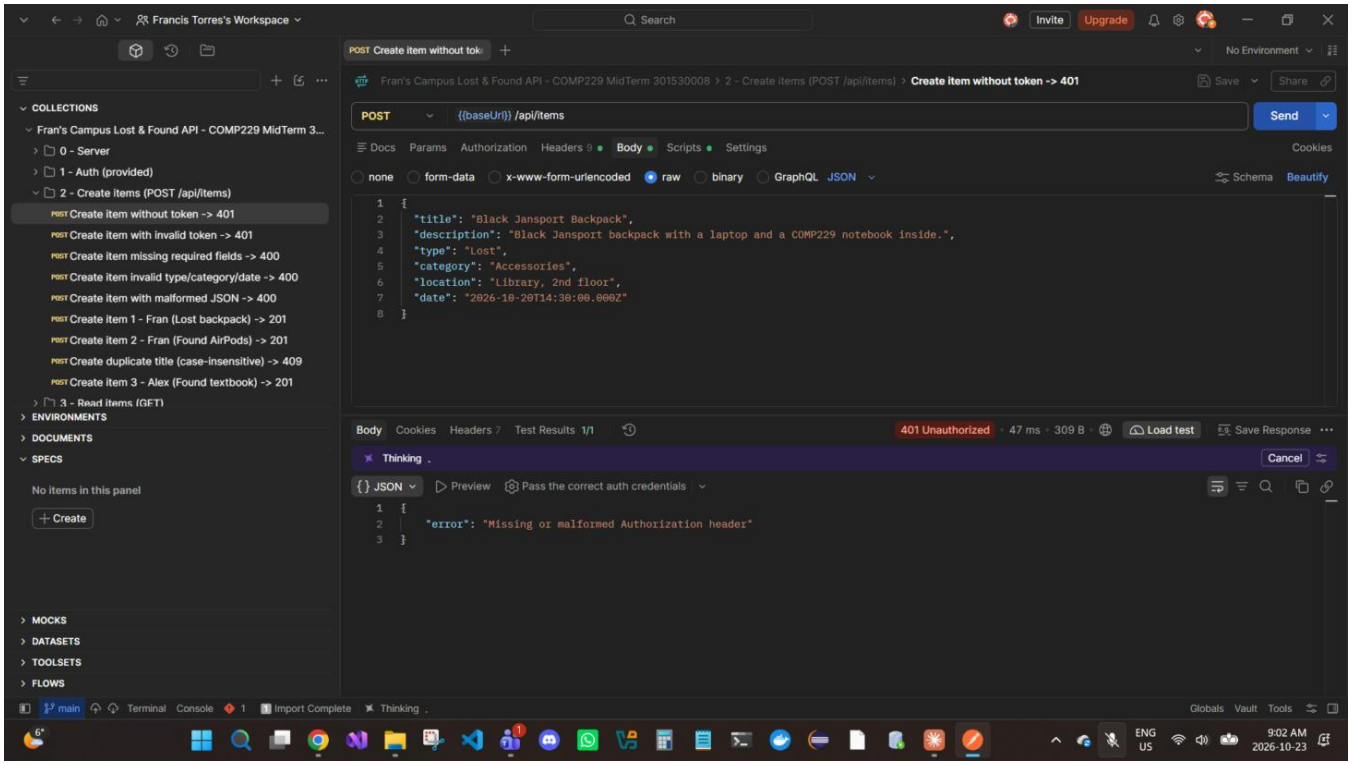


Figure 9 — `POST /api/items` — Create without an Authorization header → **401 Unauthorized**

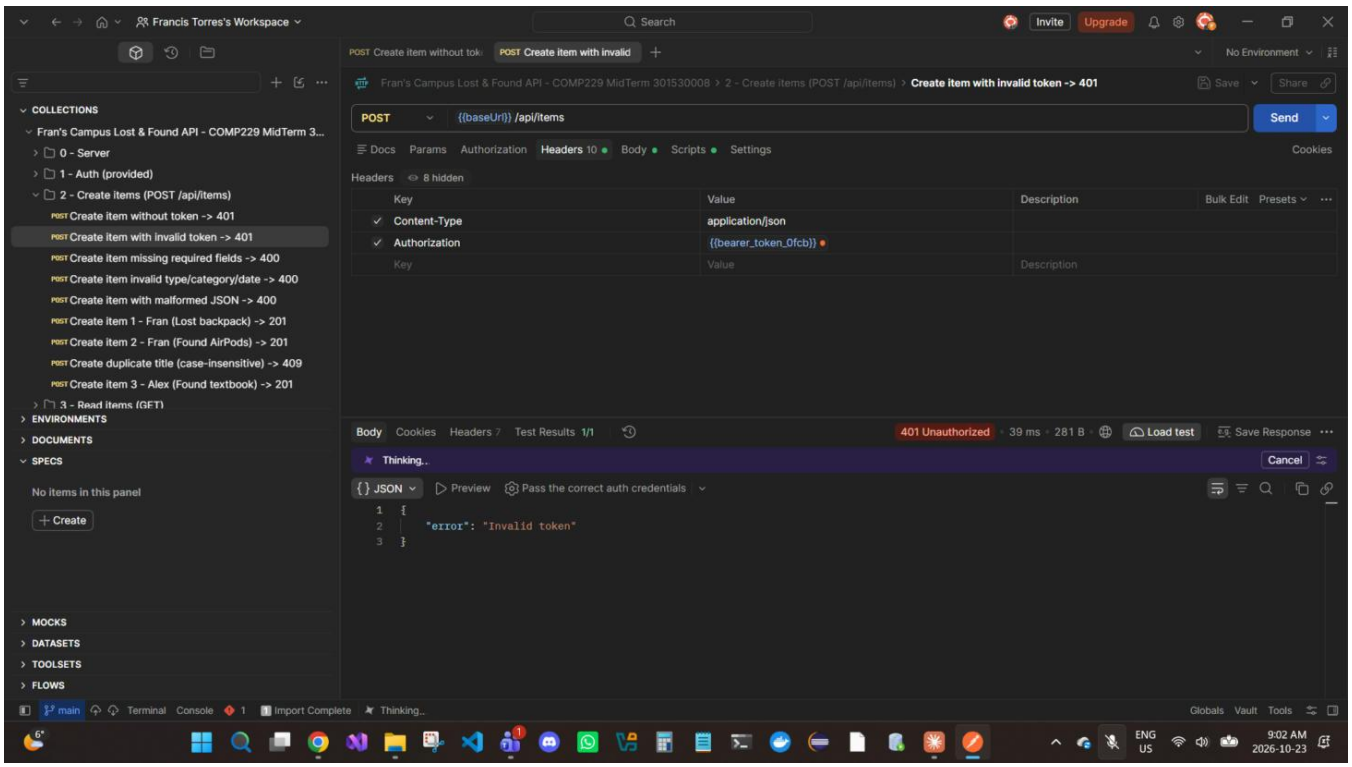


Figure 10 — `POST /api/items` — Create with a forged token ("Bearer not.a.real.token"; Postman masks the literal value as a vault variable) → **401 Unauthorized**

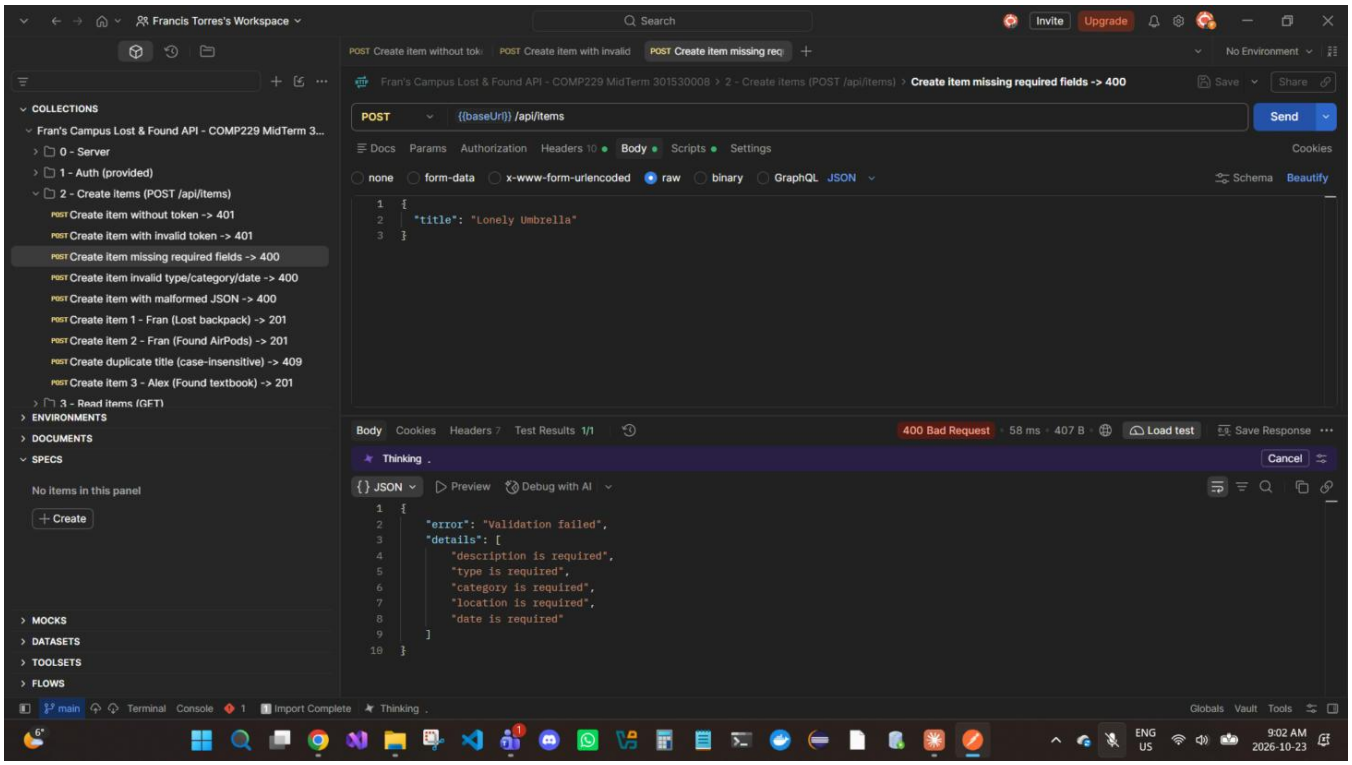


Figure 11 — POST `/api/items` — Only a title is sent; every missing required field is reported → **400 Bad Request**

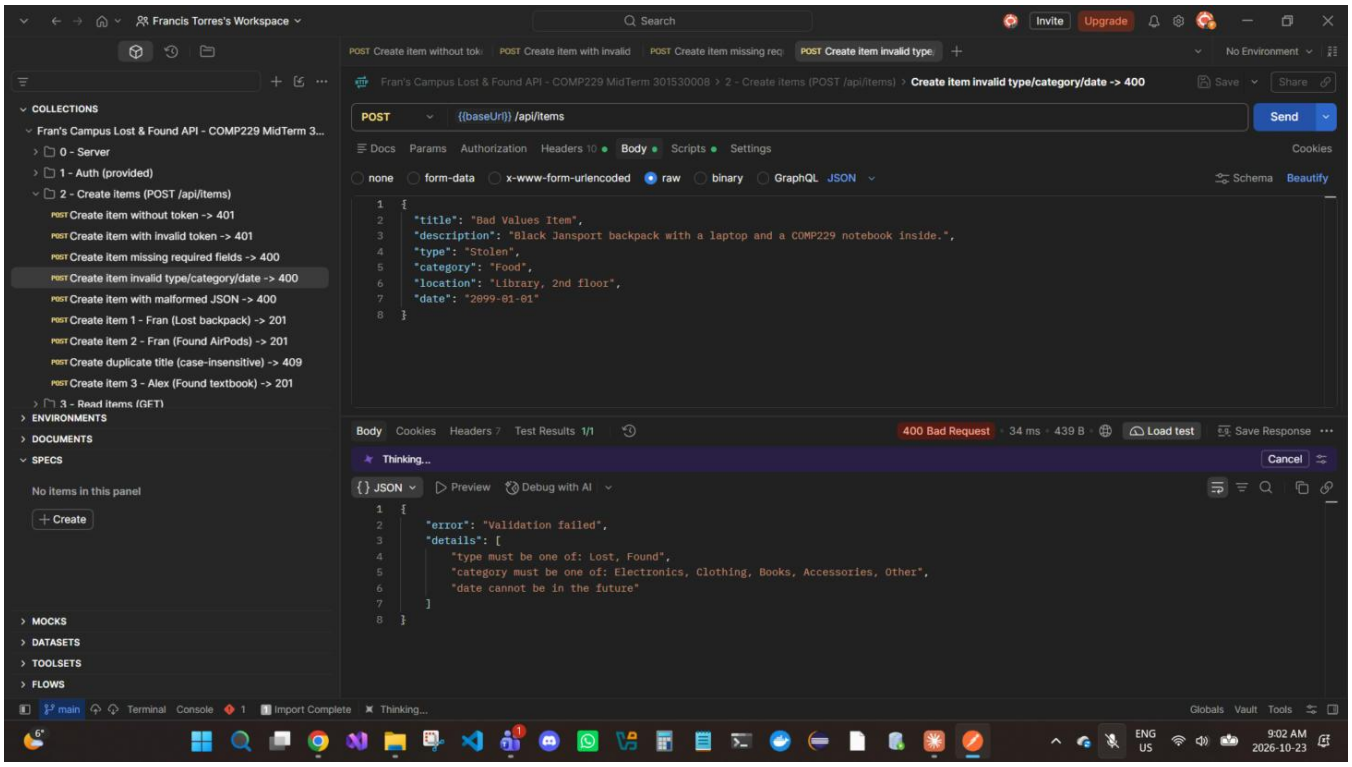


Figure 12 — POST `/api/items` — Invalid type, category and a future date are rejected → **400 Bad Request**

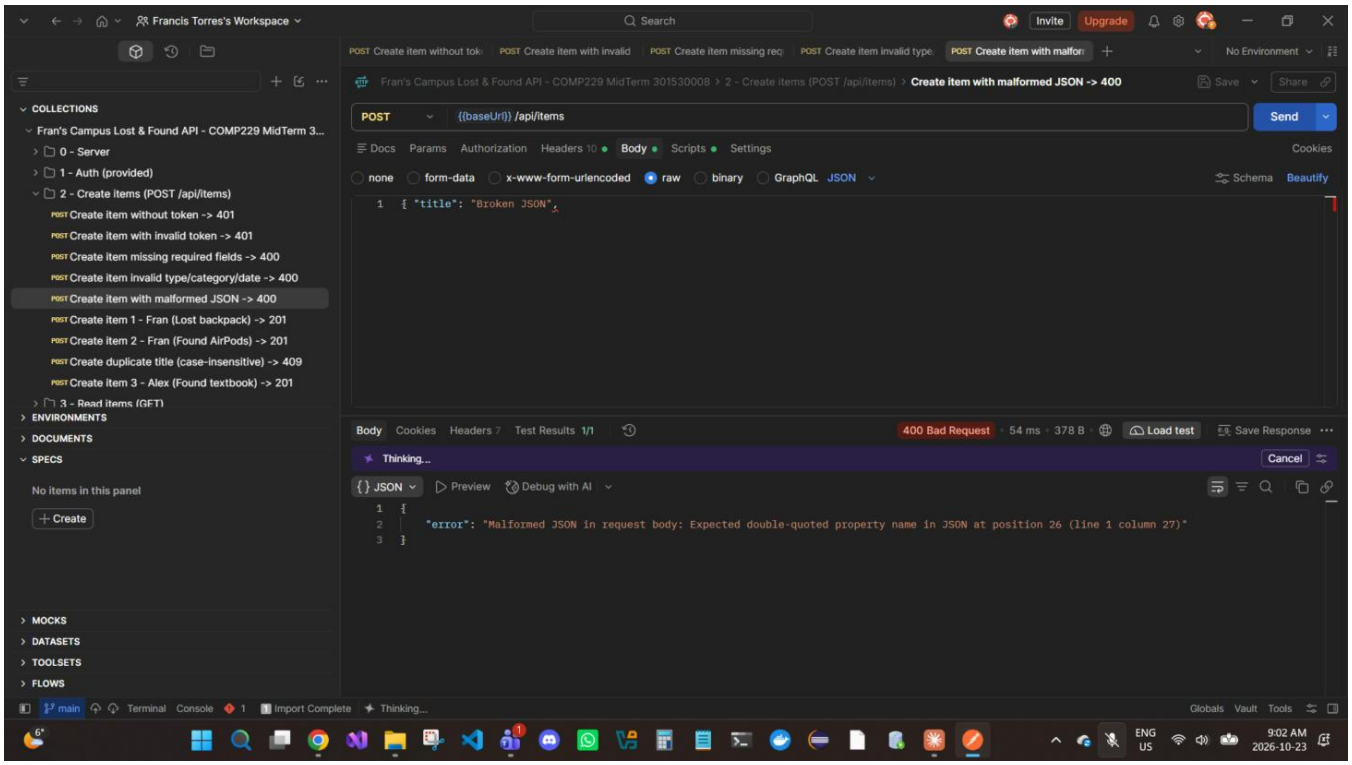


Figure 13 — `POST /api/items` — Malformed JSON body → **400 Bad Request**

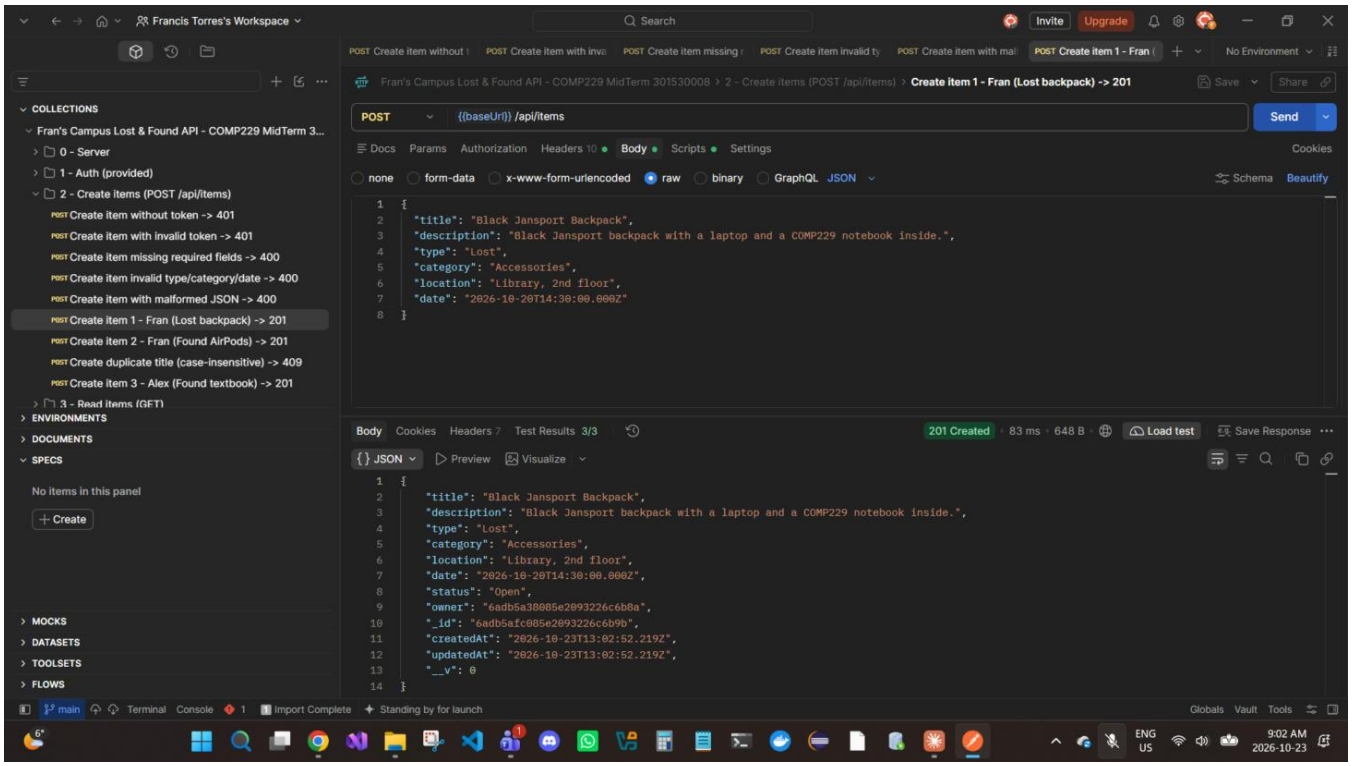


Figure 14 — `POST /api/items` — Fran creates "Black Jansport Backpack"; owner is set from the JWT and status defaults to Open → **201 Created**

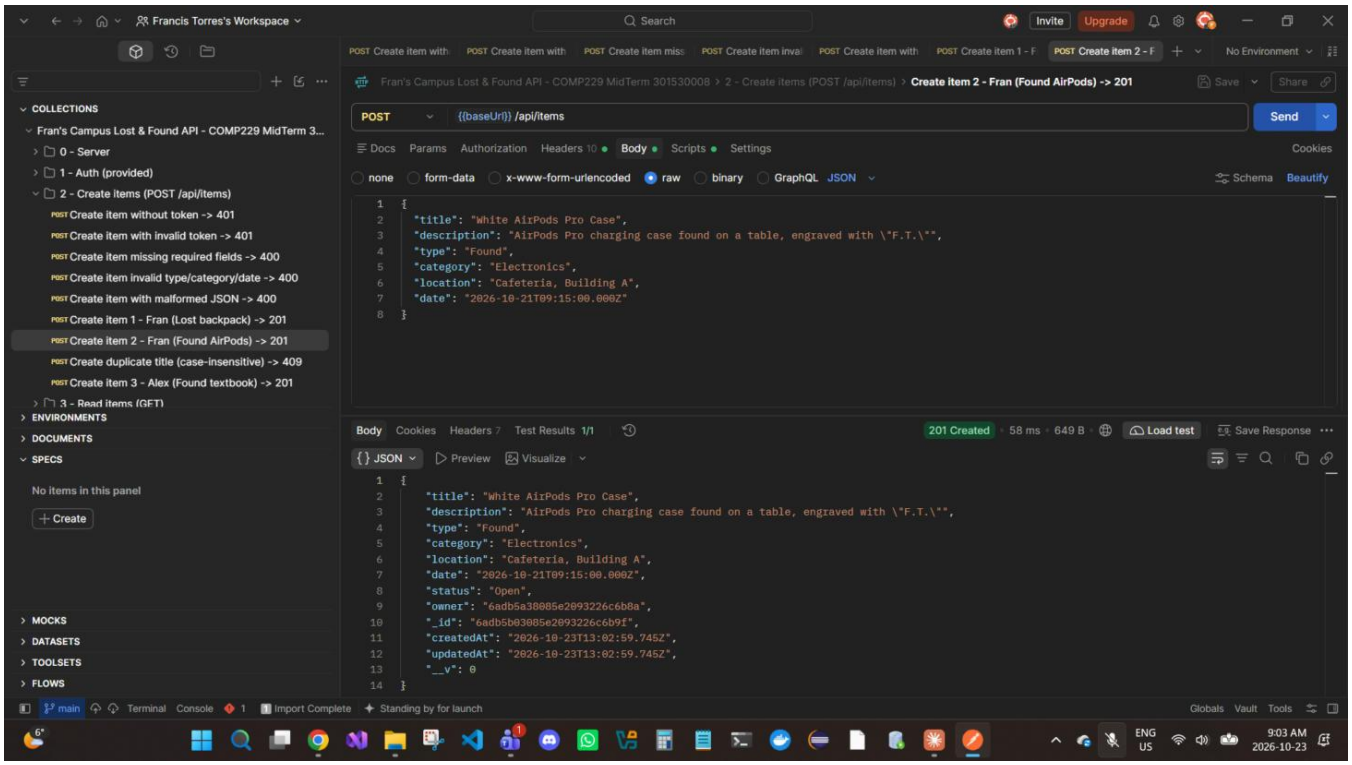


Figure 15 — `POST /api/items` — Fran creates "White AirPods Pro Case" → **201 Created**

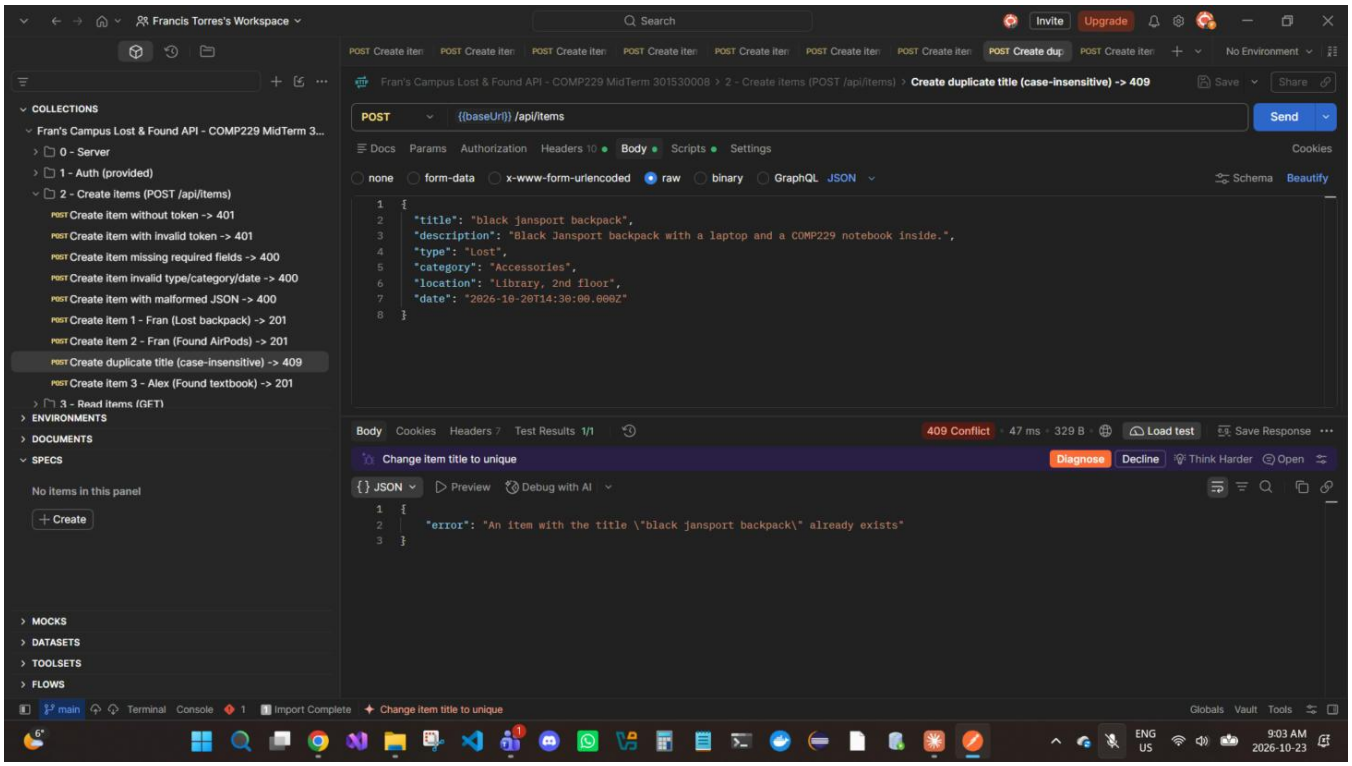


Figure 16 — `POST /api/items` — Duplicate title in different case ("black jansport backpack") is rejected; title is the unique field → **409 Conflict**

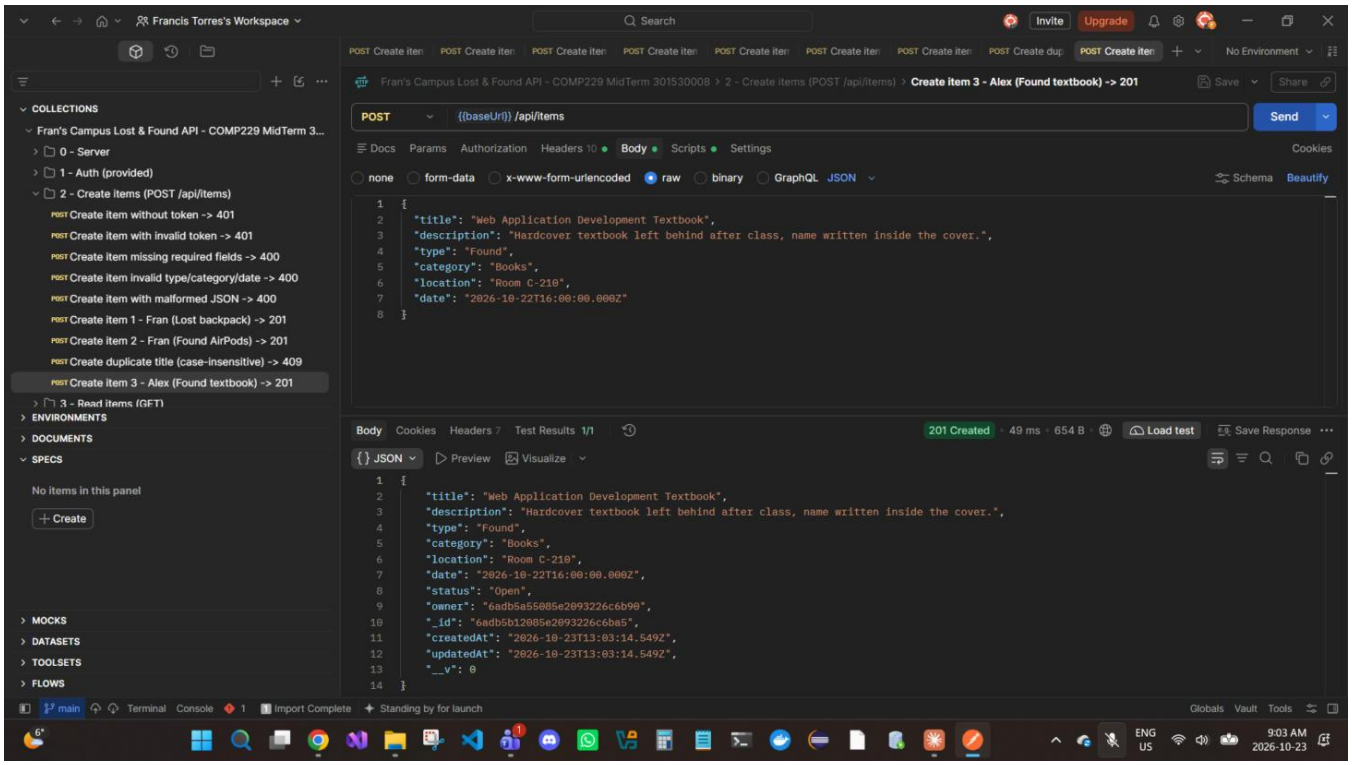


Figure 17 — `POST /api/items` — Alex creates "Web Application Development Textbook" → 201 Created

MongoDB evidence — Create items

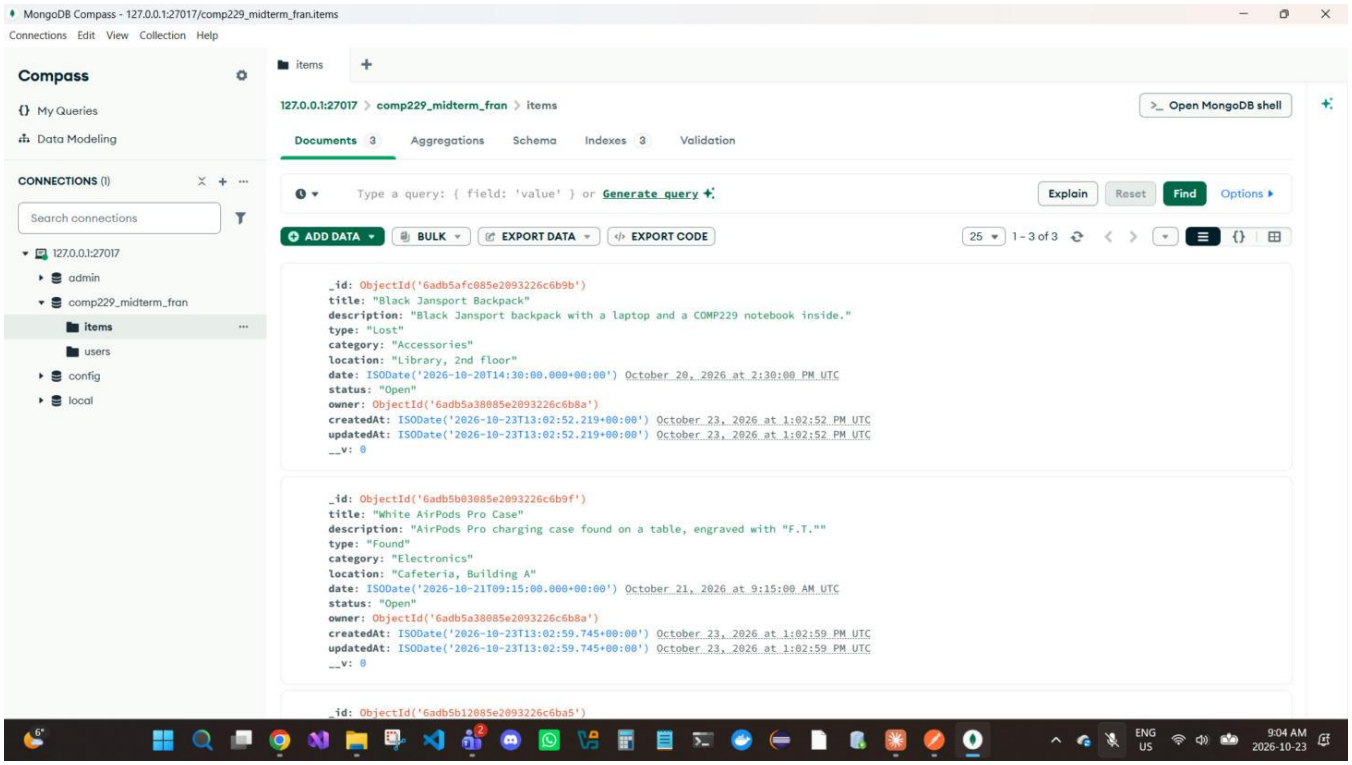


Figure 18 — MongoDB Compass: comp229_midterm_fran.items now holds the 3 created documents (1–3 of 3) with owner ObjectIds and timestamps.

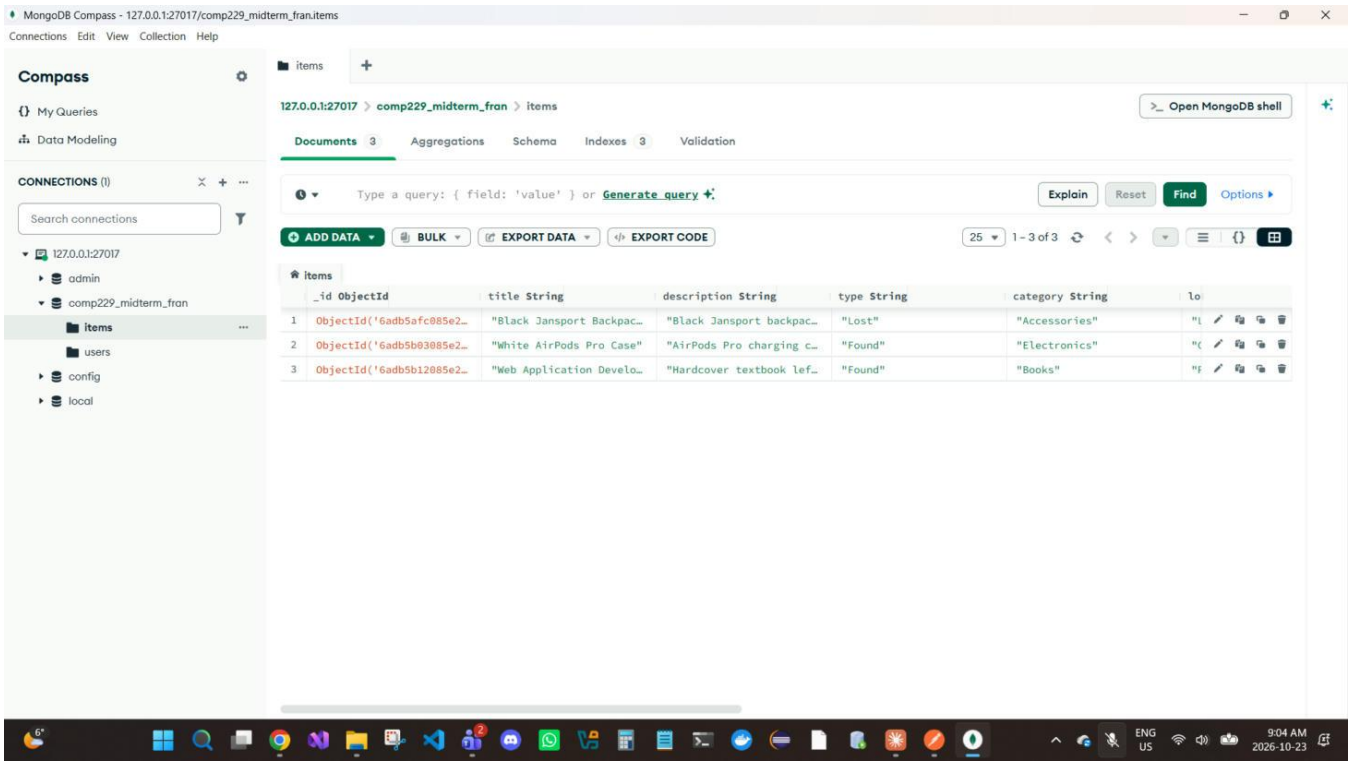


Figure 19 — MongoDB Compass table view of the same 3 documents.

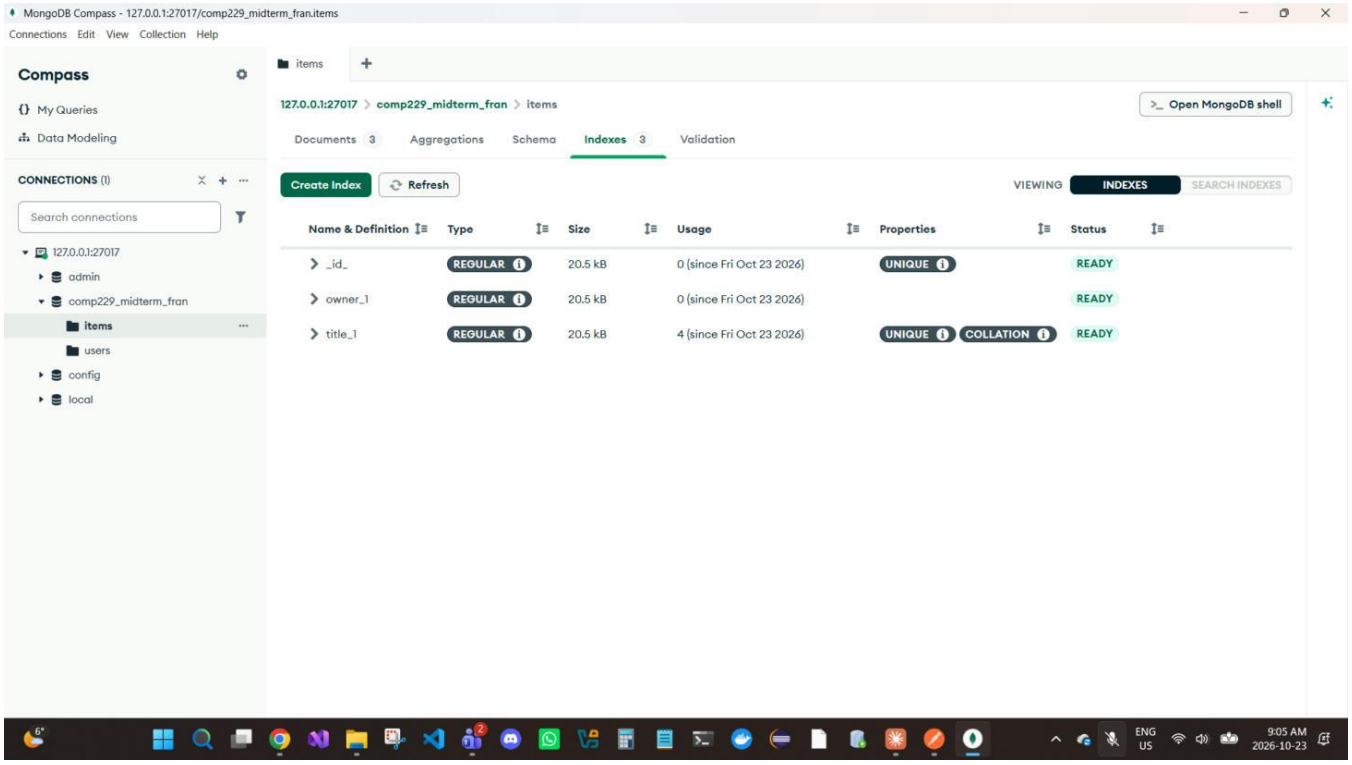


Figure 20 — Indexes tab: title_1 is UNIQUE with a case-insensitive COLLATION (prevents duplicate items); owner_1 speeds up /mine queries.

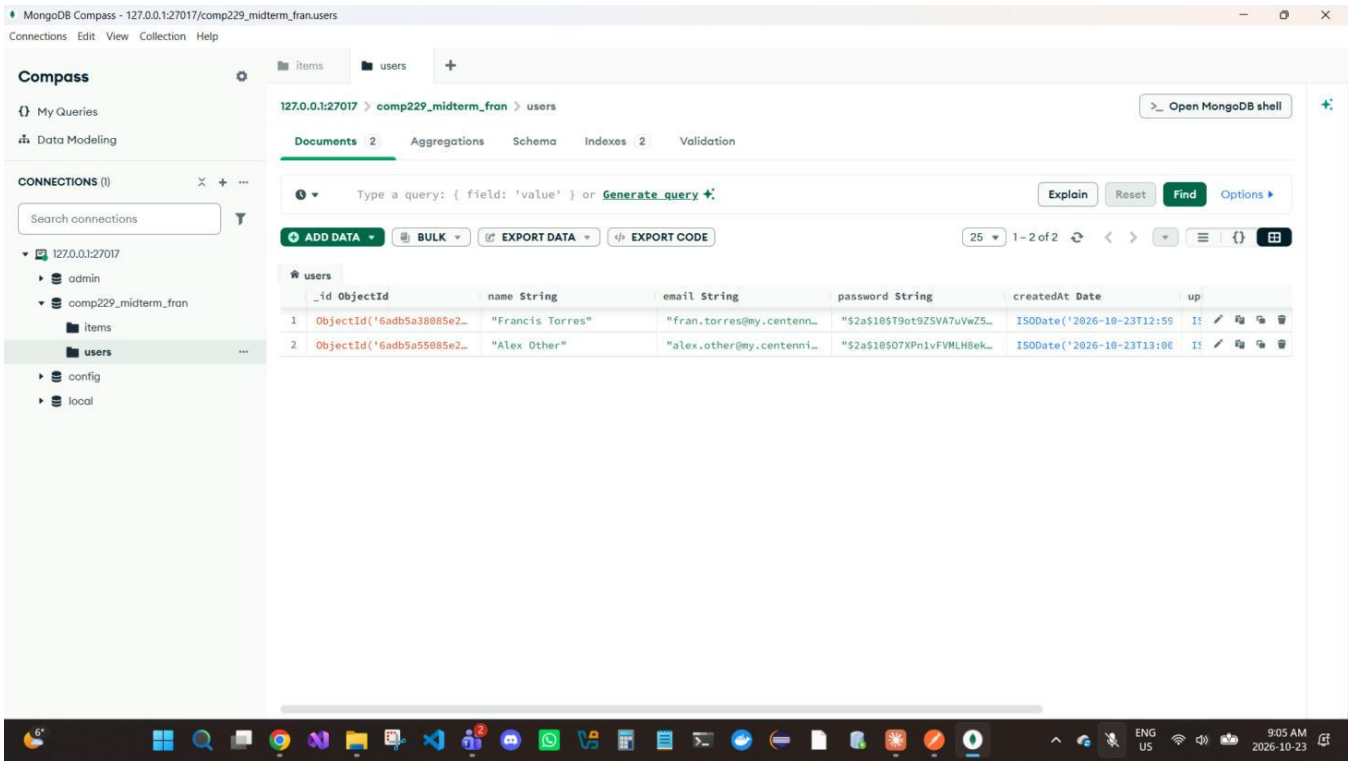


Figure 21 — users collection: both test users, with bcrypt-hashed passwords ($2a$10$...), never plain text.

4. Read items — GET (public and Bearer)

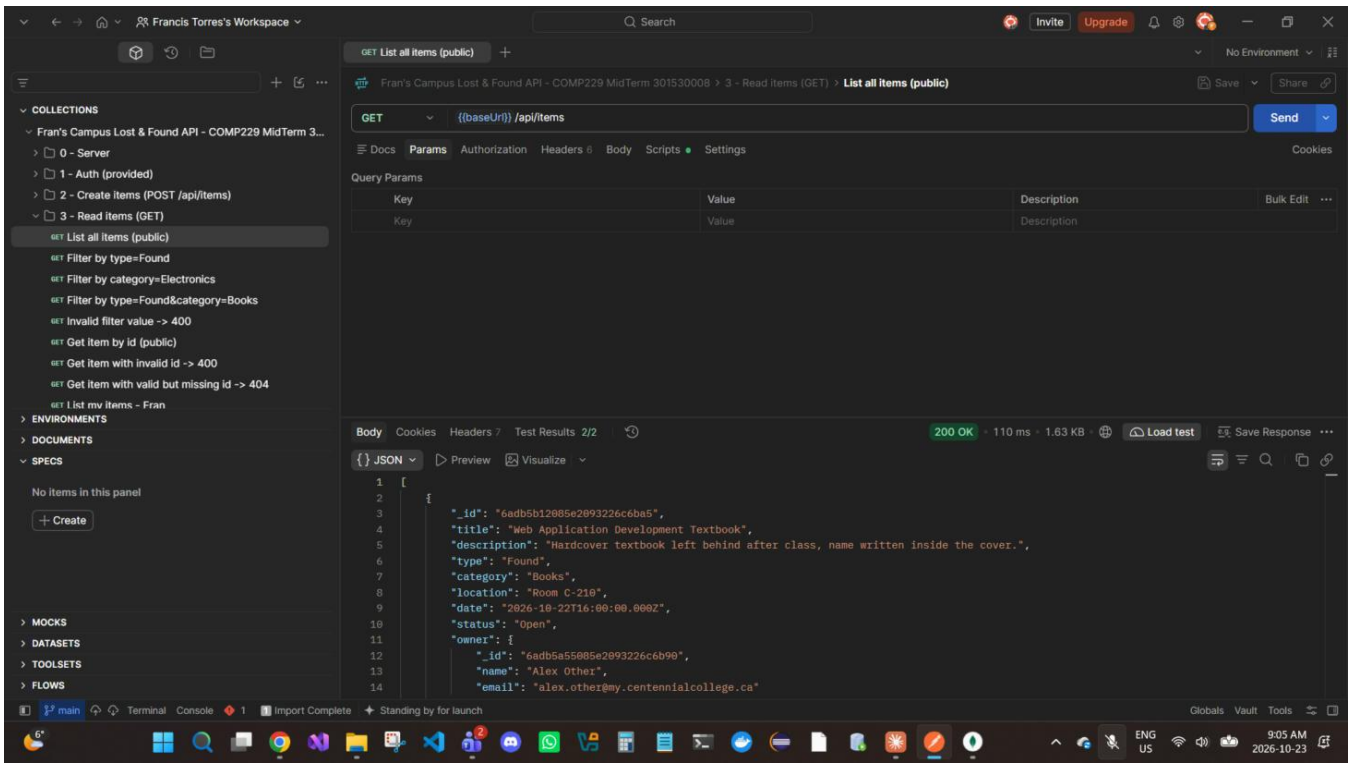


Figure 22 — `GET /api/items` — List all items (public); test asserts 3 items → **200 OK**

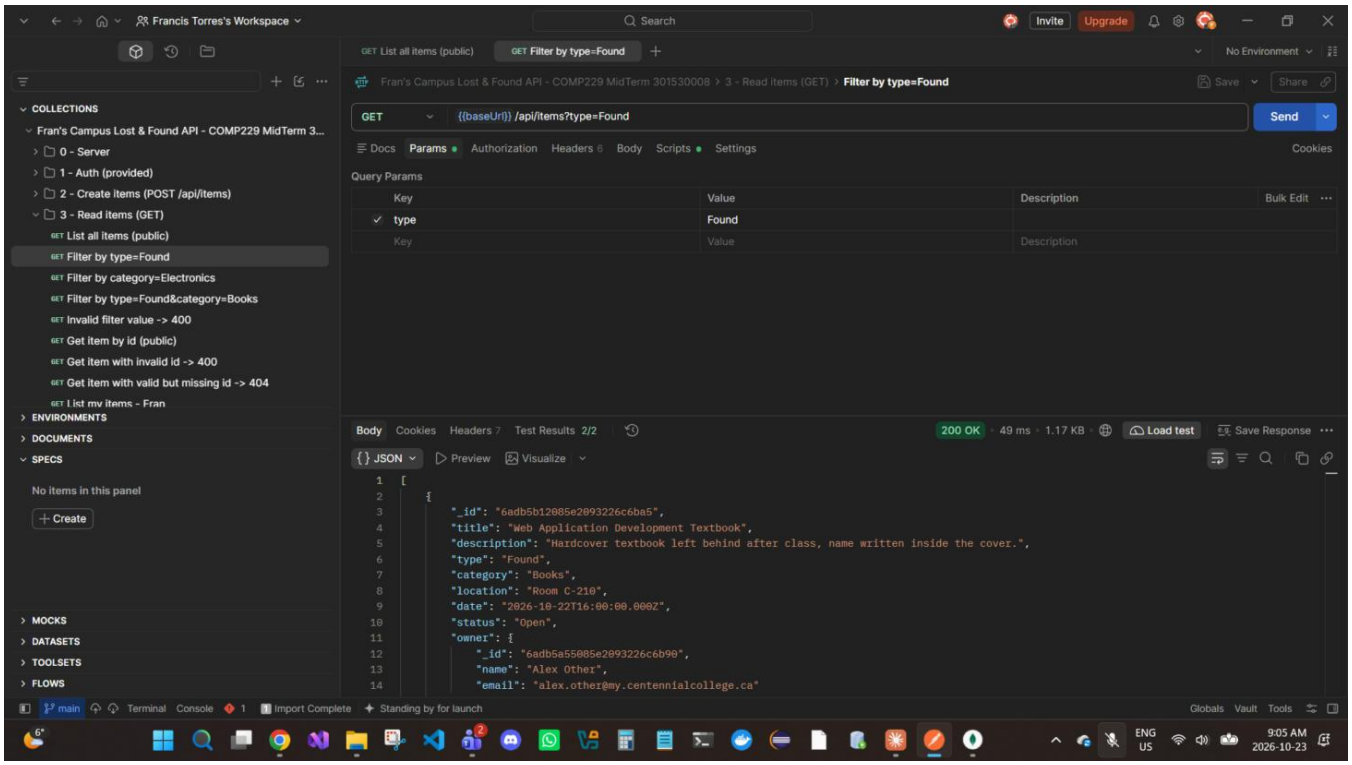


Figure 23 — `GET /api/items?type=Found` — Filter by type; every result is Found → **200 OK**

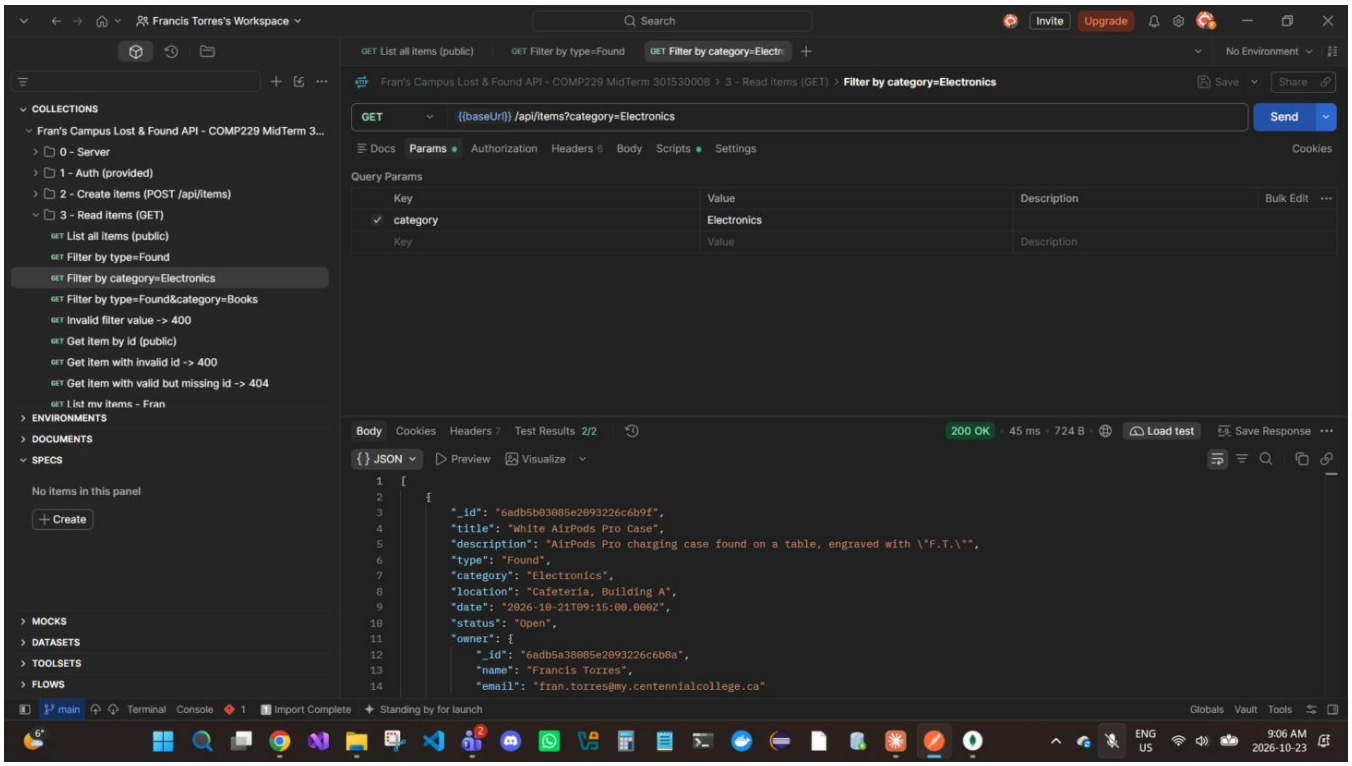


Figure 24 — `GET /api/items?category=Electronics` — Filter by category → **200 OK**

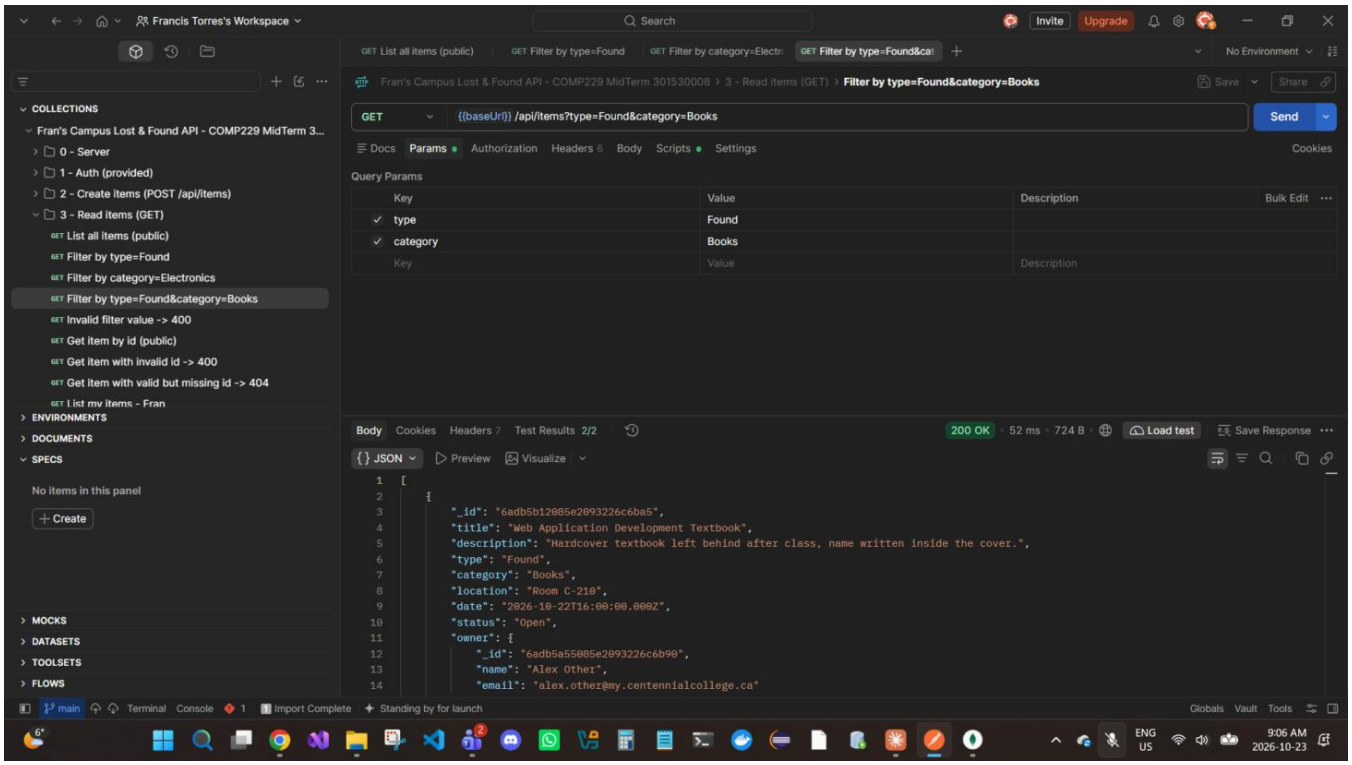


Figure 25 — `GET /api/items?type=Found&category=Books` — Combined filters; exactly 1 match → **200 OK**

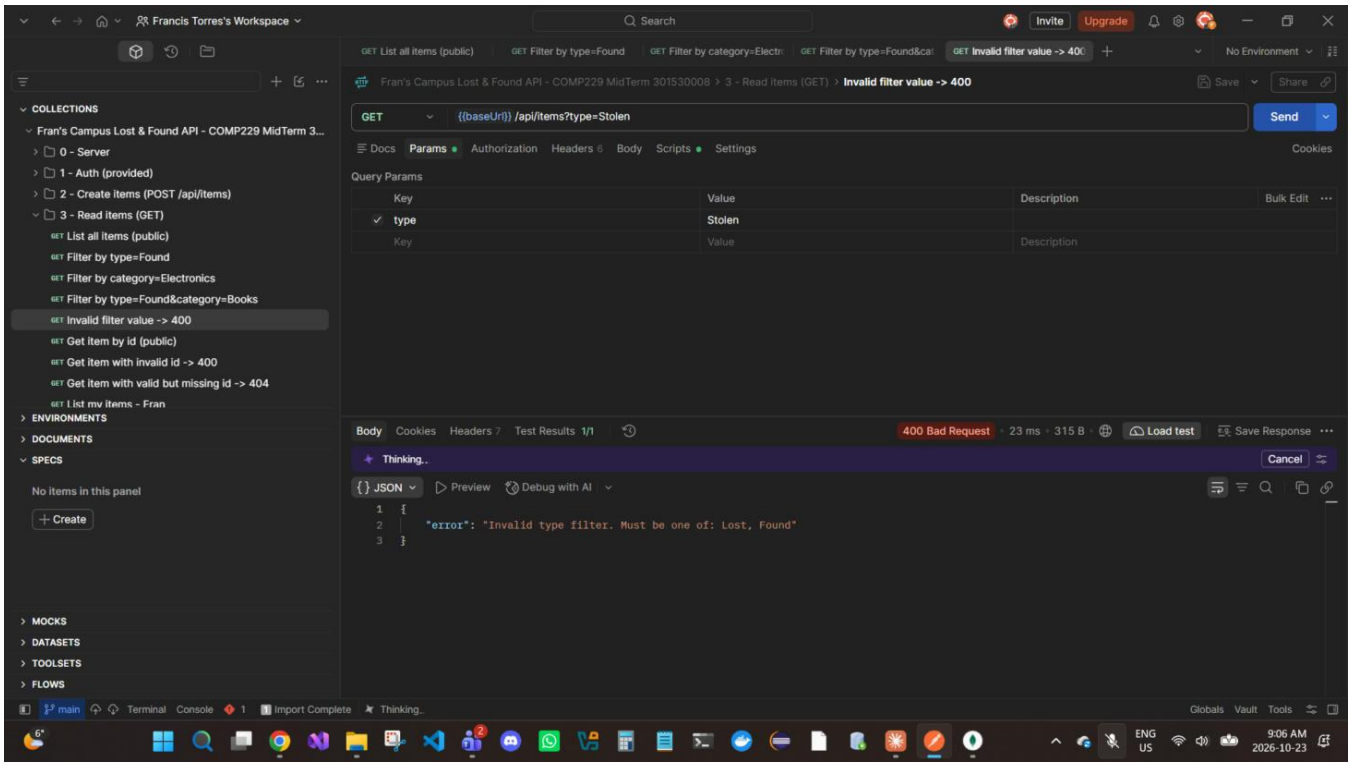


Figure 26 — `GET /api/items?type=Stolen` — Unknown filter value → **400 Bad Request**

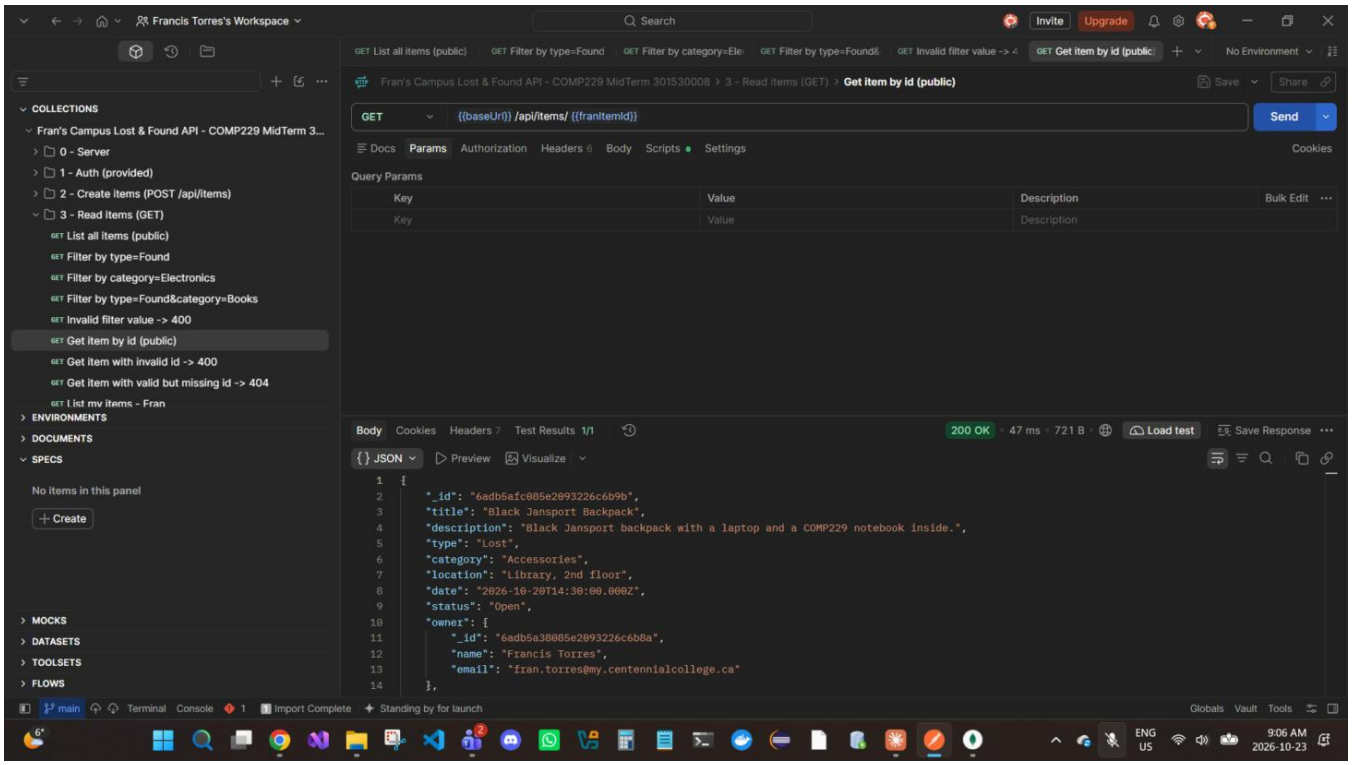


Figure 27 — `GET /api/items/{{franItemId}}` — Get a single item by id (public, owner populated) → **200 OK**

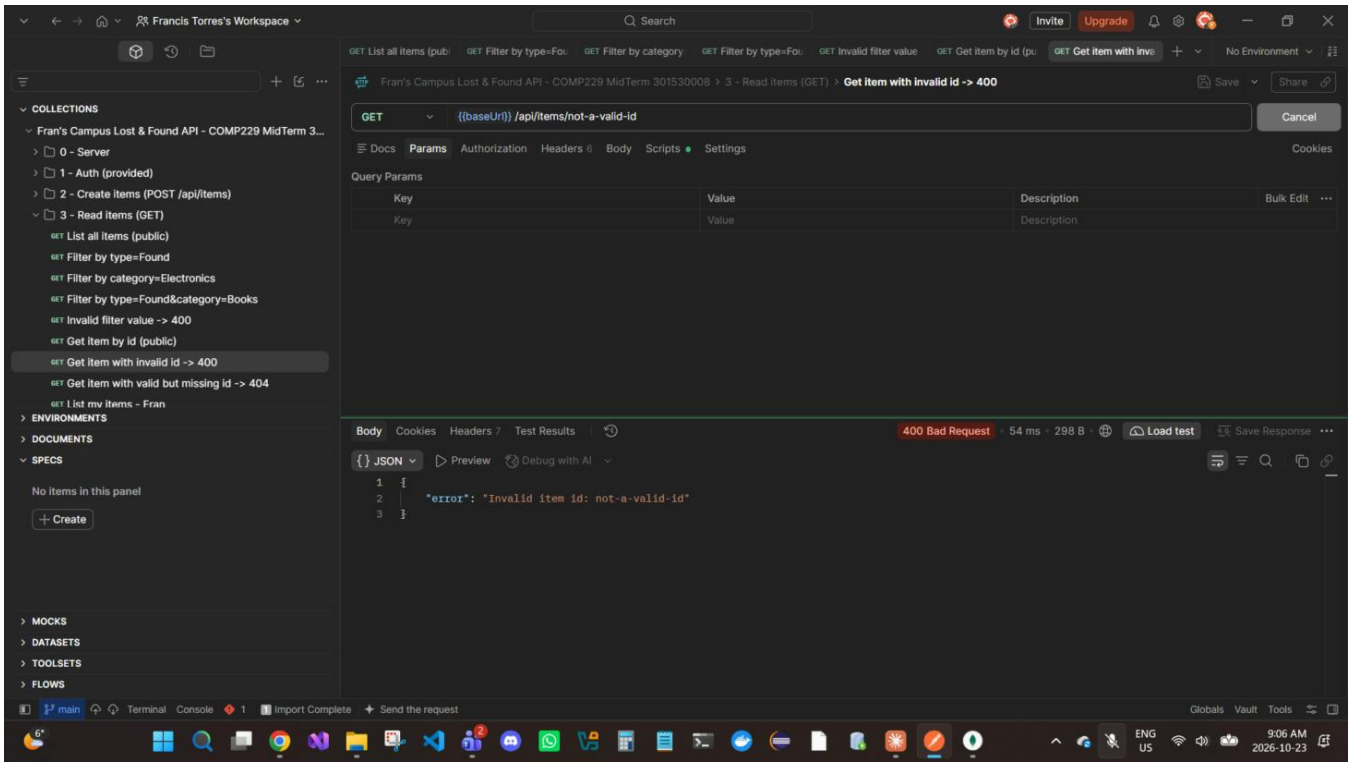


Figure 28 — `GET /api/items/not-a-valid-id` — Malformed ObjectId → **400 Bad Request**

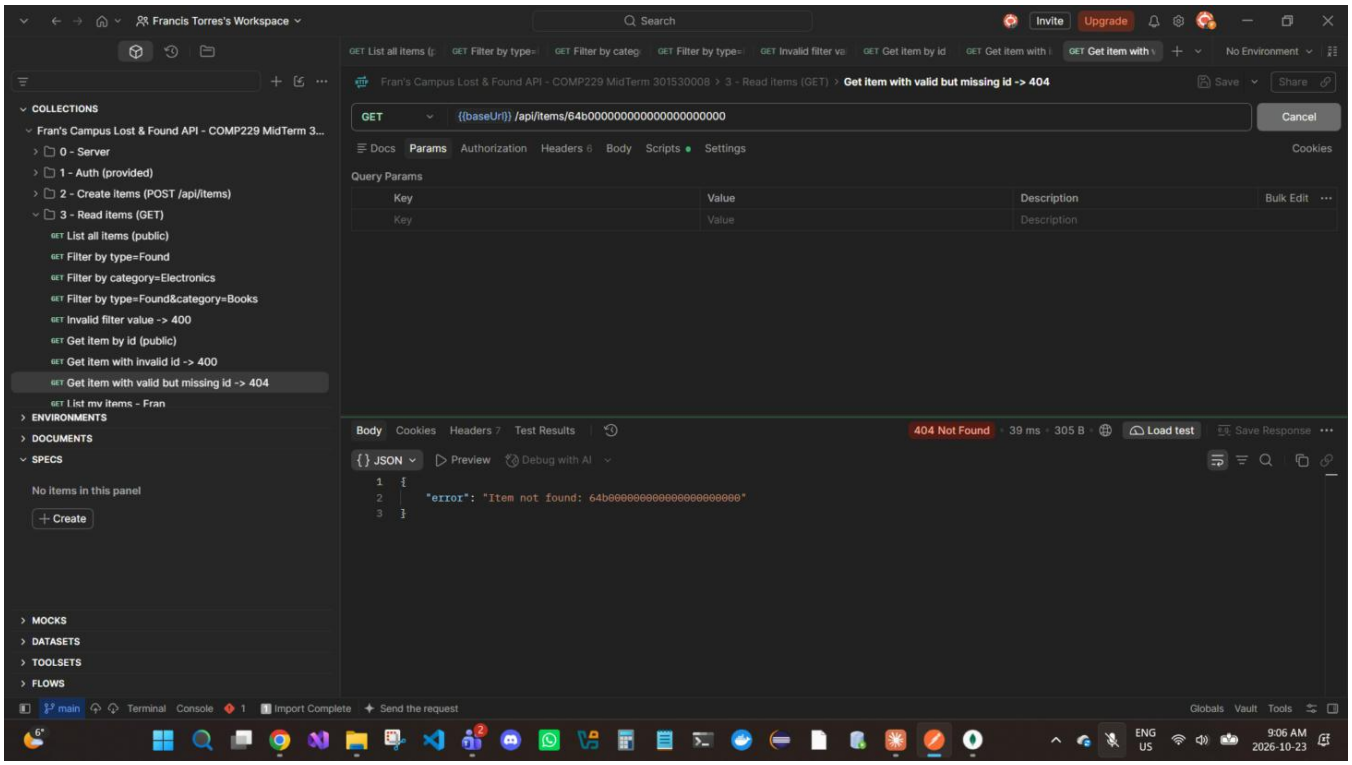


Figure 29 — `GET /api/items/64b000000000000000000000` — Valid ObjectId that does not exist → **404 Not Found**

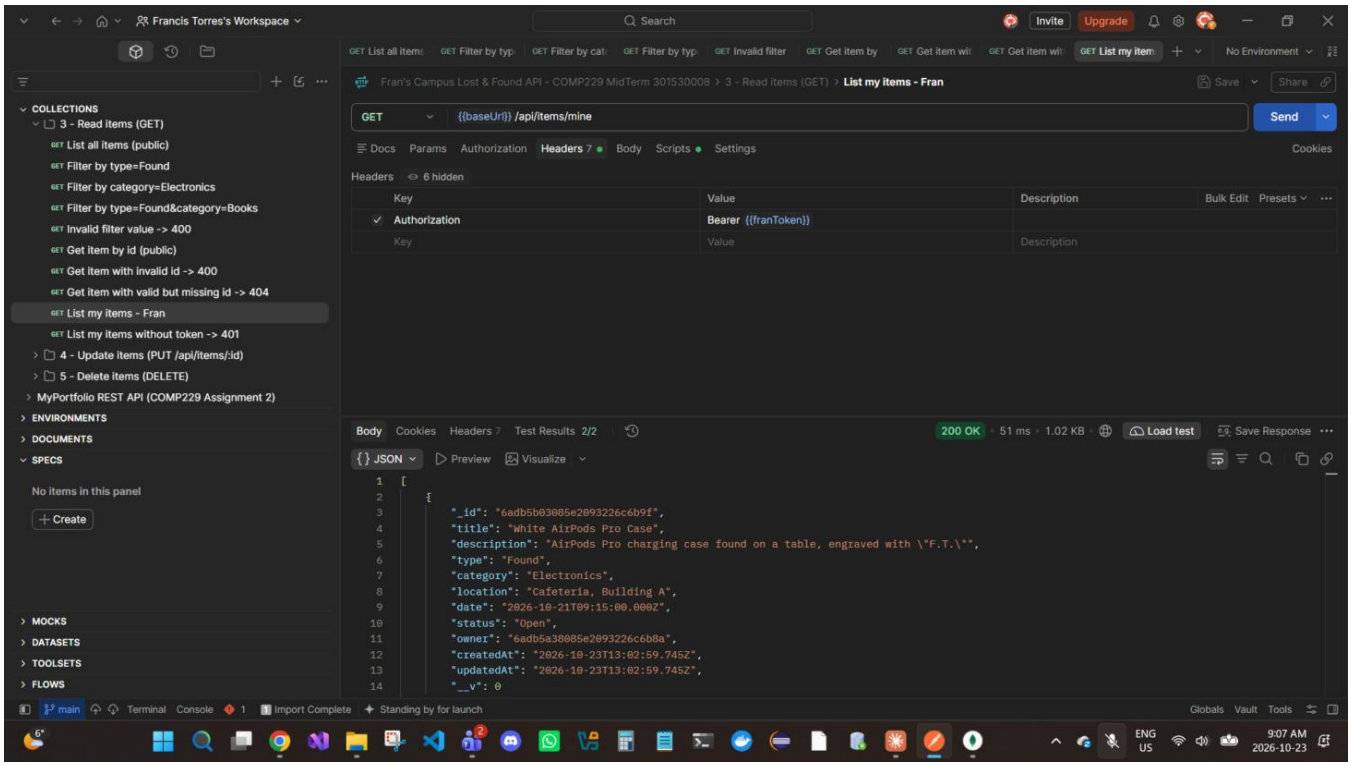


Figure 30 — `GET /api/items/mine` — Fran's items only (Bearer {{franToken}}) → **200 OK**

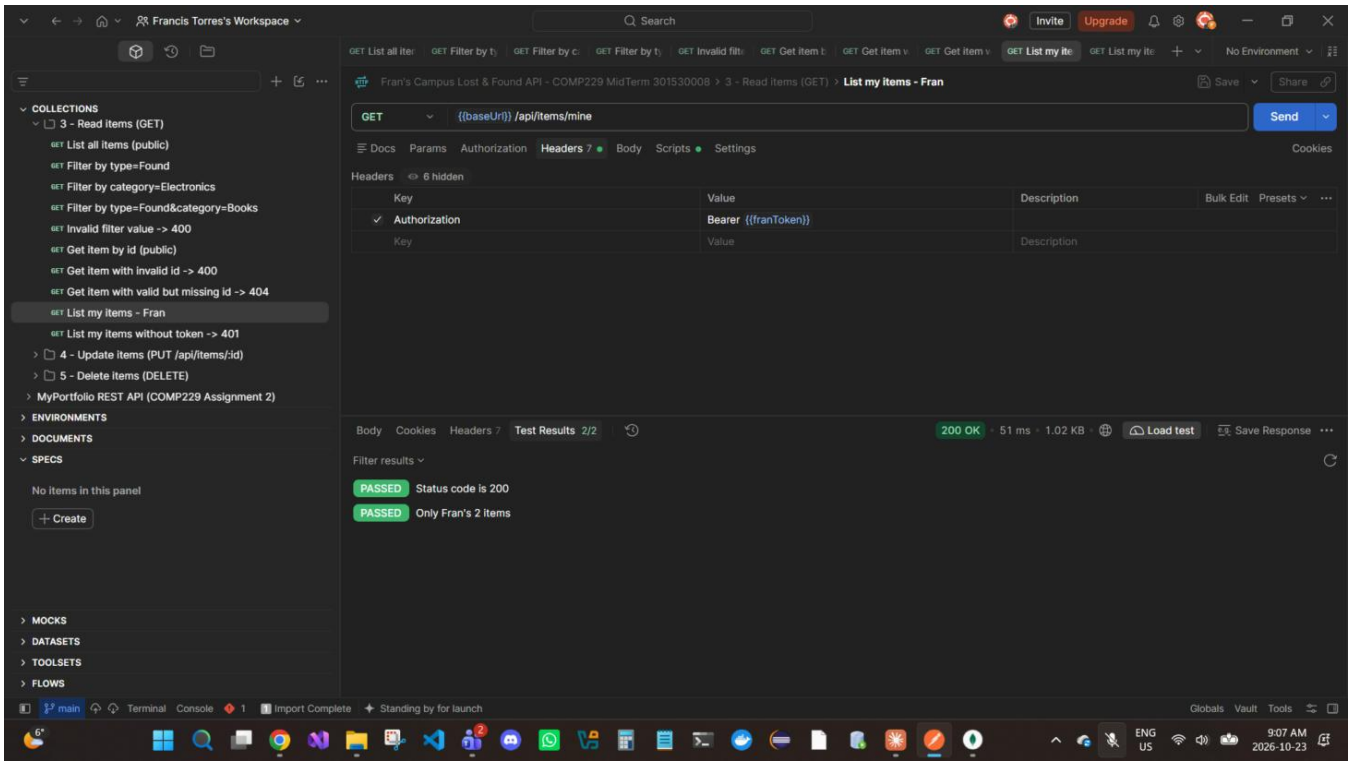


Figure 31 — `GET /api/items/mine` — Postman Test Results: "Only Fran's 2 items" PASSED (Alex's item is excluded) → **200 OK**

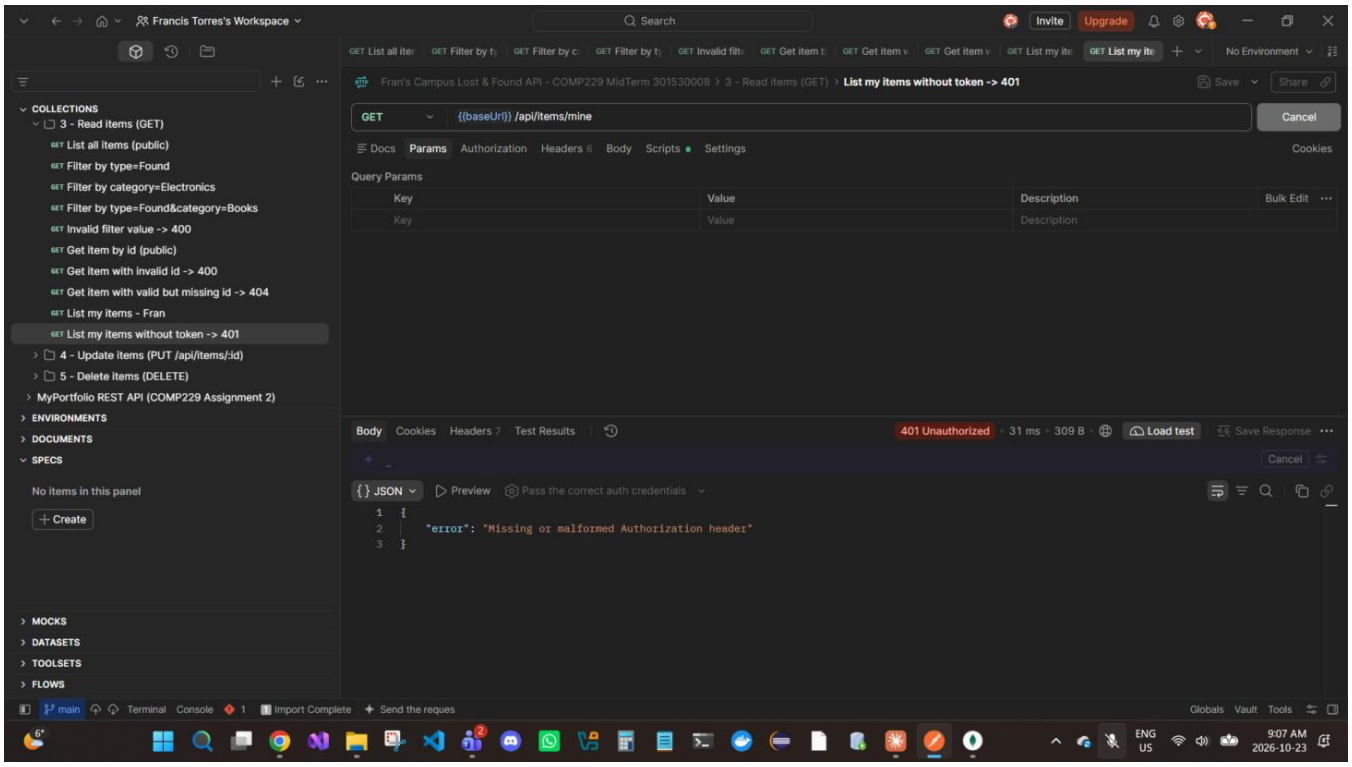


Figure 32 — `GET /api/items/mine` — Protected route without a token → **401 Unauthorized**

5. Update items — PUT /api/items/:id (Bearer, owner only)

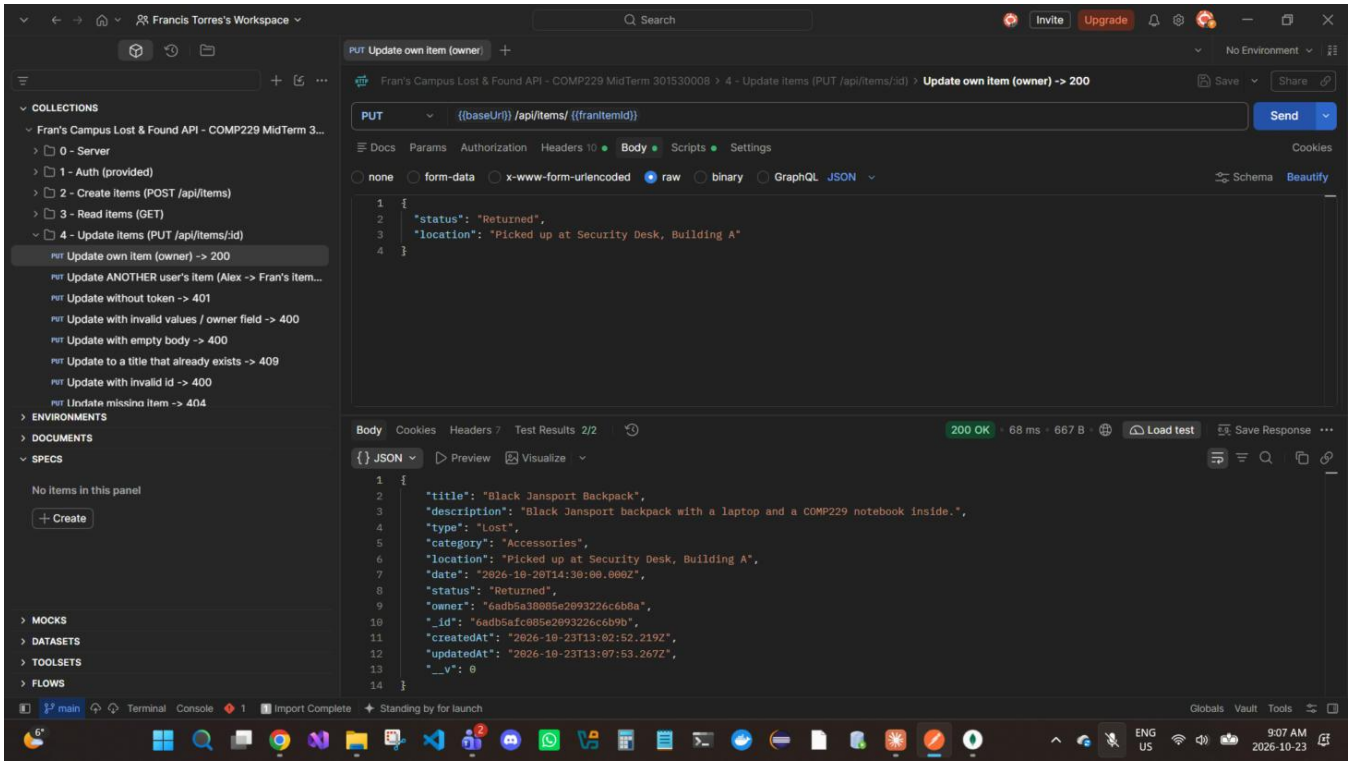


Figure 33 — PUT `/api/items/{{franItemId}}` — Owner (Fran) marks the backpack Returned and changes its location → **200 OK**

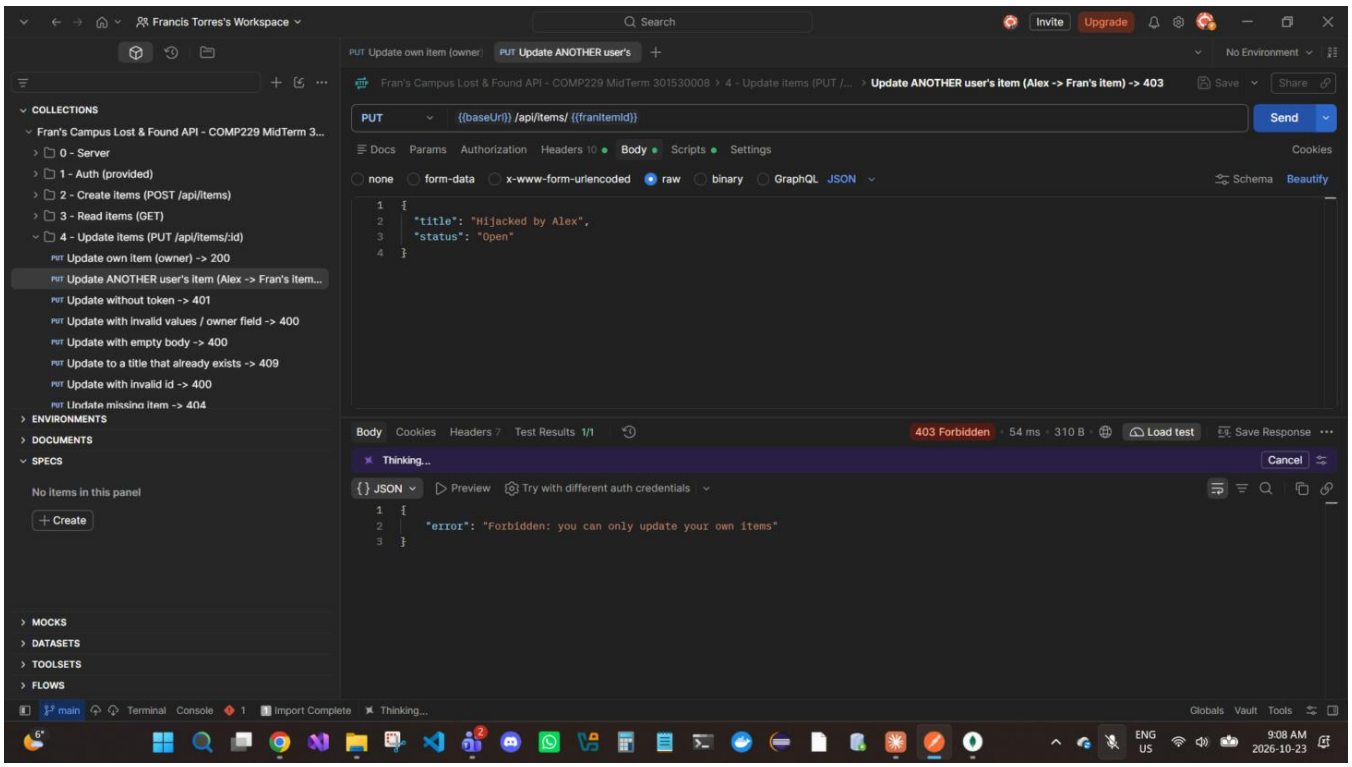


Figure 34 — PUT `/api/items/{{franItemId}}` — OWNERSHIP: Alex tries to rename Fran's item to "Hijacked by Alex" — rejected → **403 Forbidden**

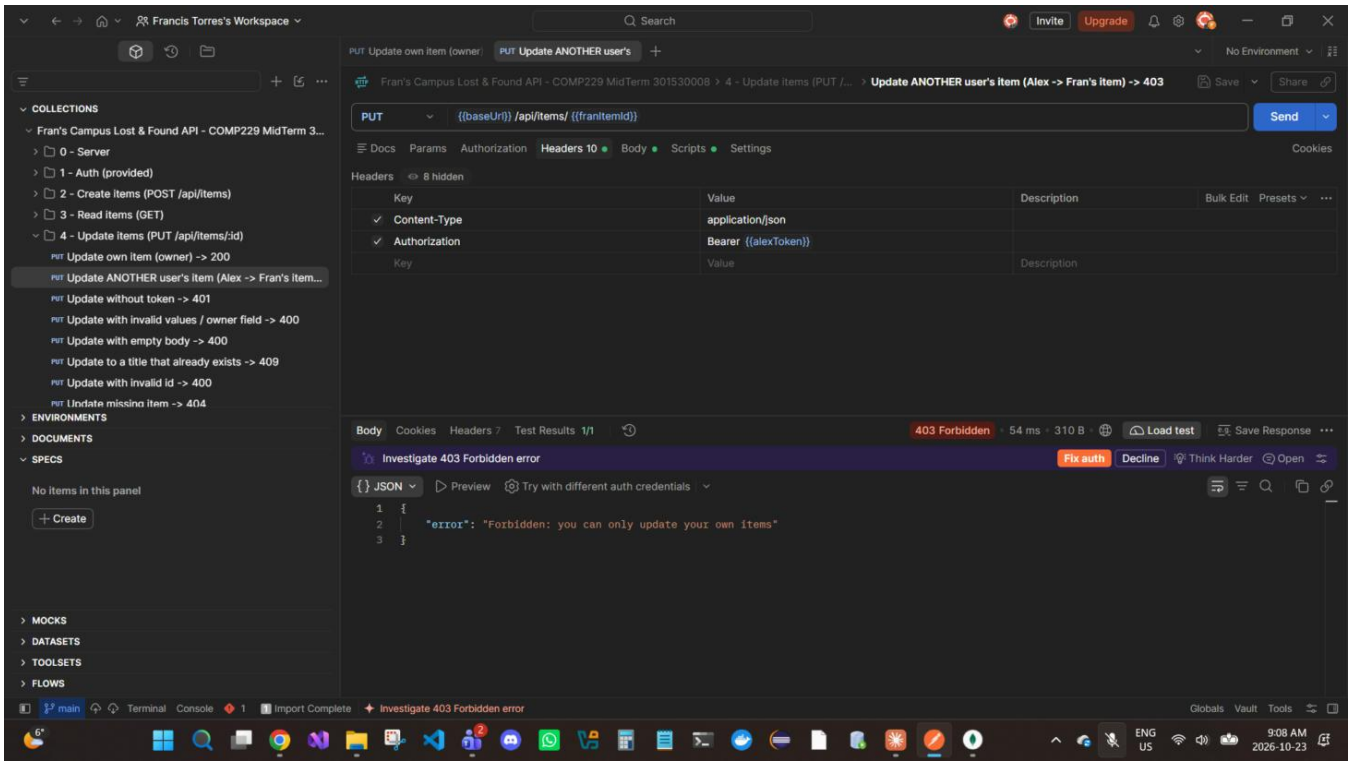


Figure 35 — PUT `/api/items/{{franItemId}}` — Same request, Headers tab: it is sent with Alex's token (Bearer {{alexToken}}}) → **403 Forbidden**

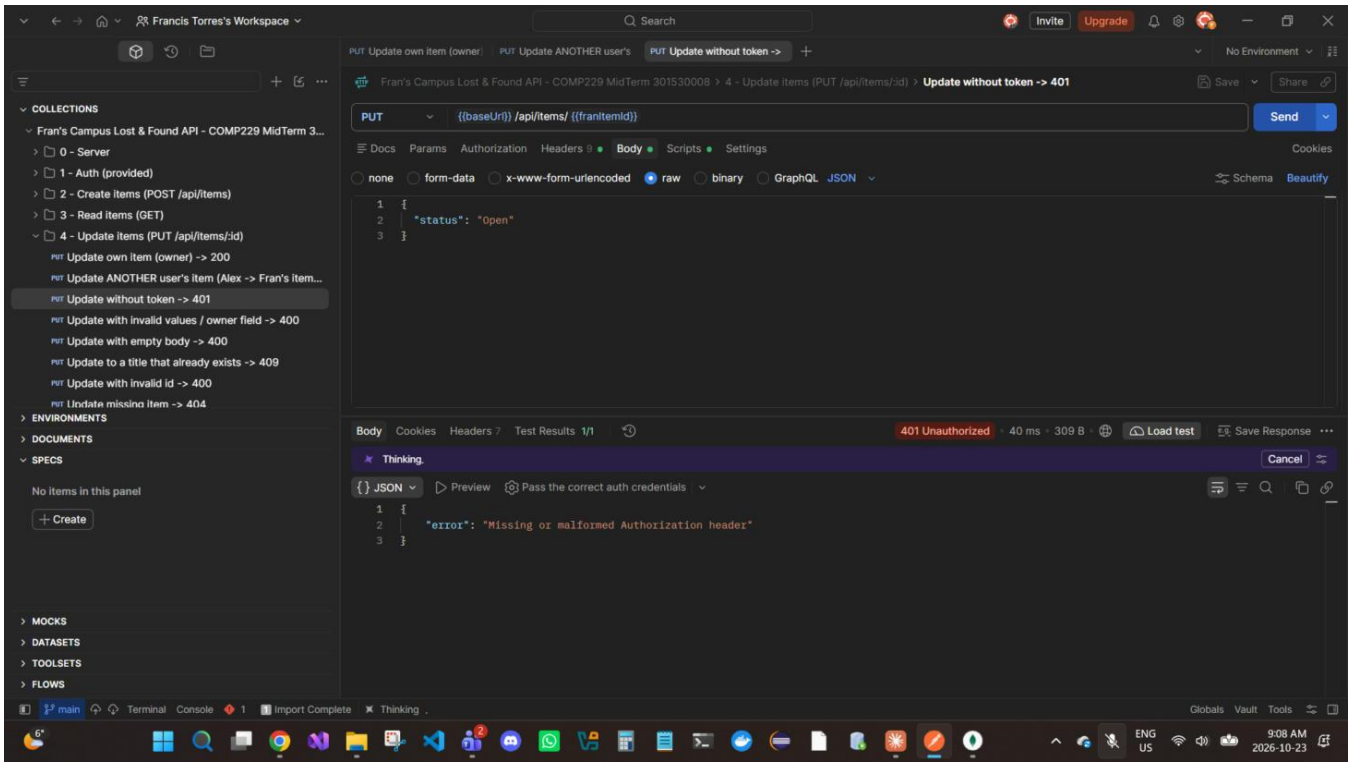


Figure 36 — PUT `/api/items/{{franItemId}}` — Update without a token → **401 Unauthorized**

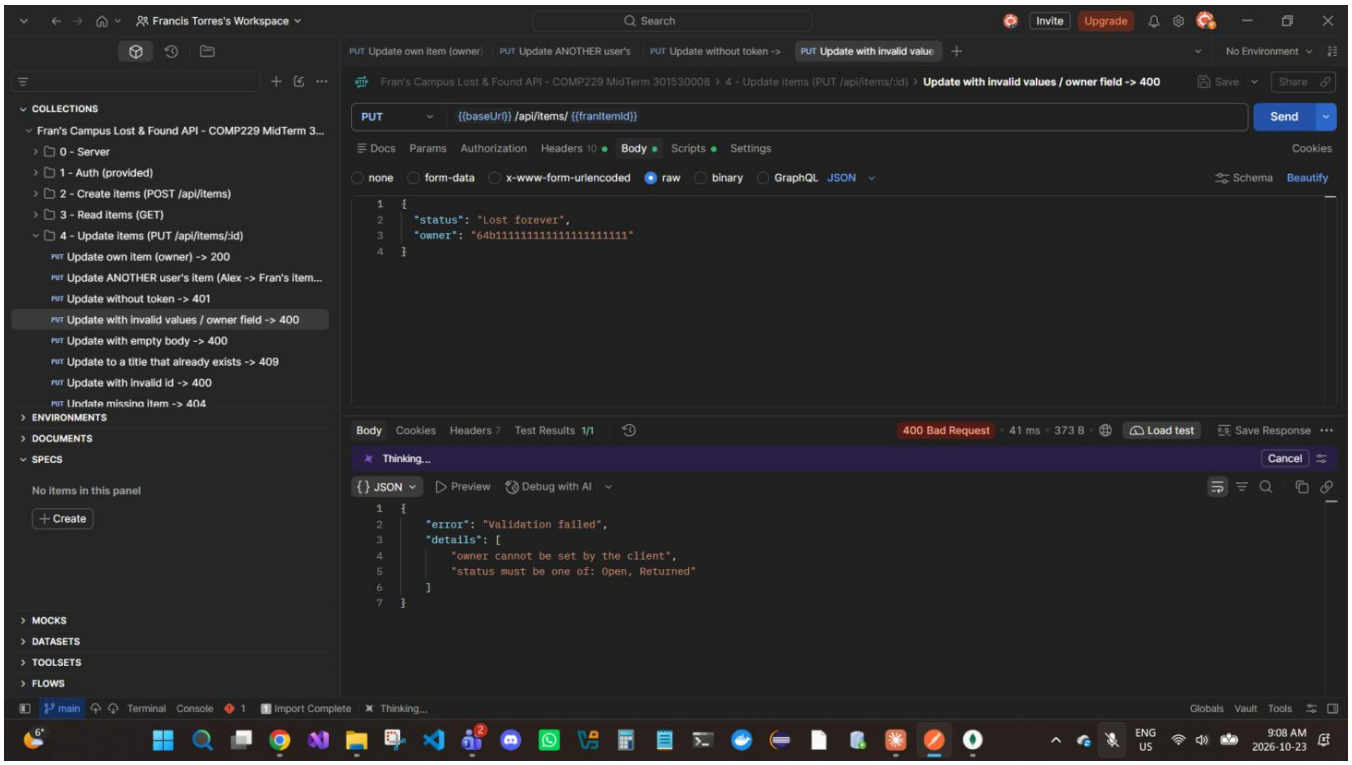


Figure 37 — PUT `/api/items/{{franItemId}}` — Invalid status and an attempt to overwrite owner → **400 Bad Request**

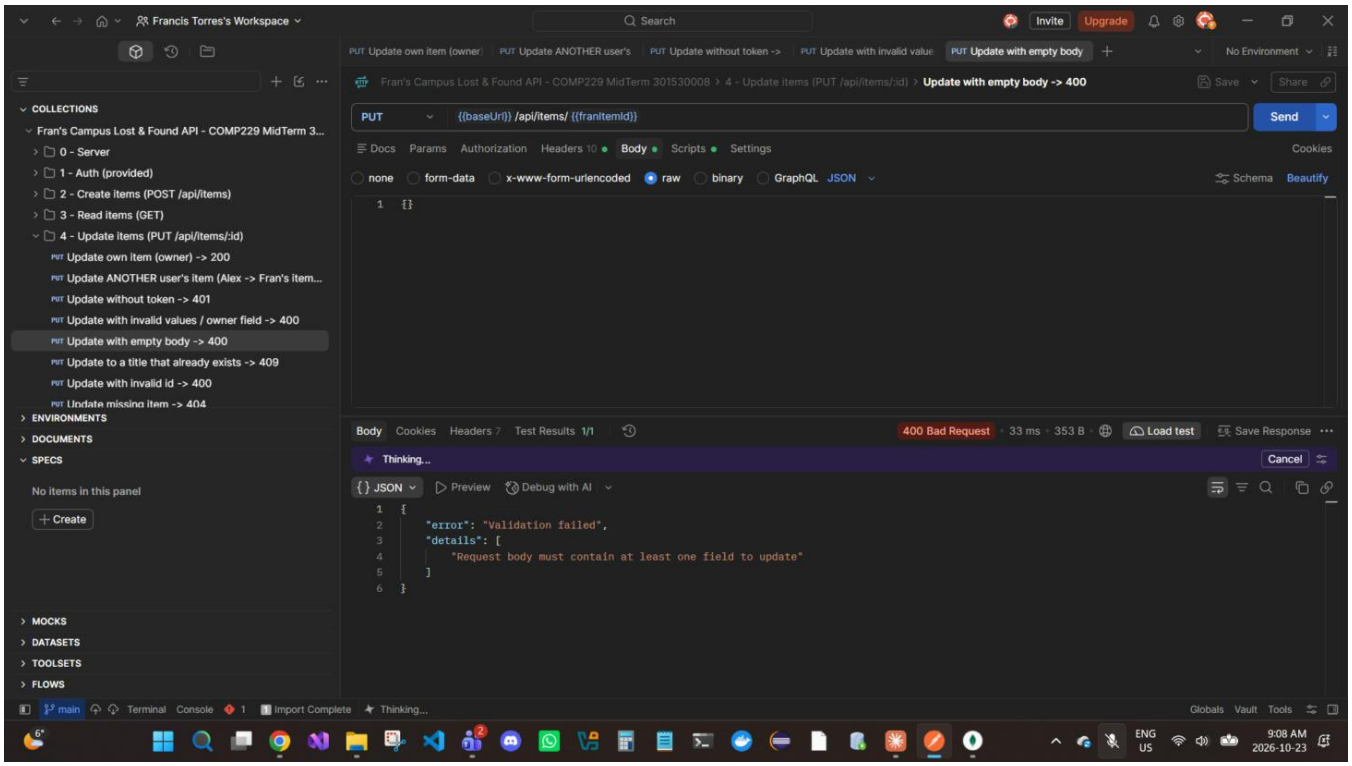


Figure 38 — PUT `/api/items/{{franItemId}}` — Empty body {} → **400 Bad Request**

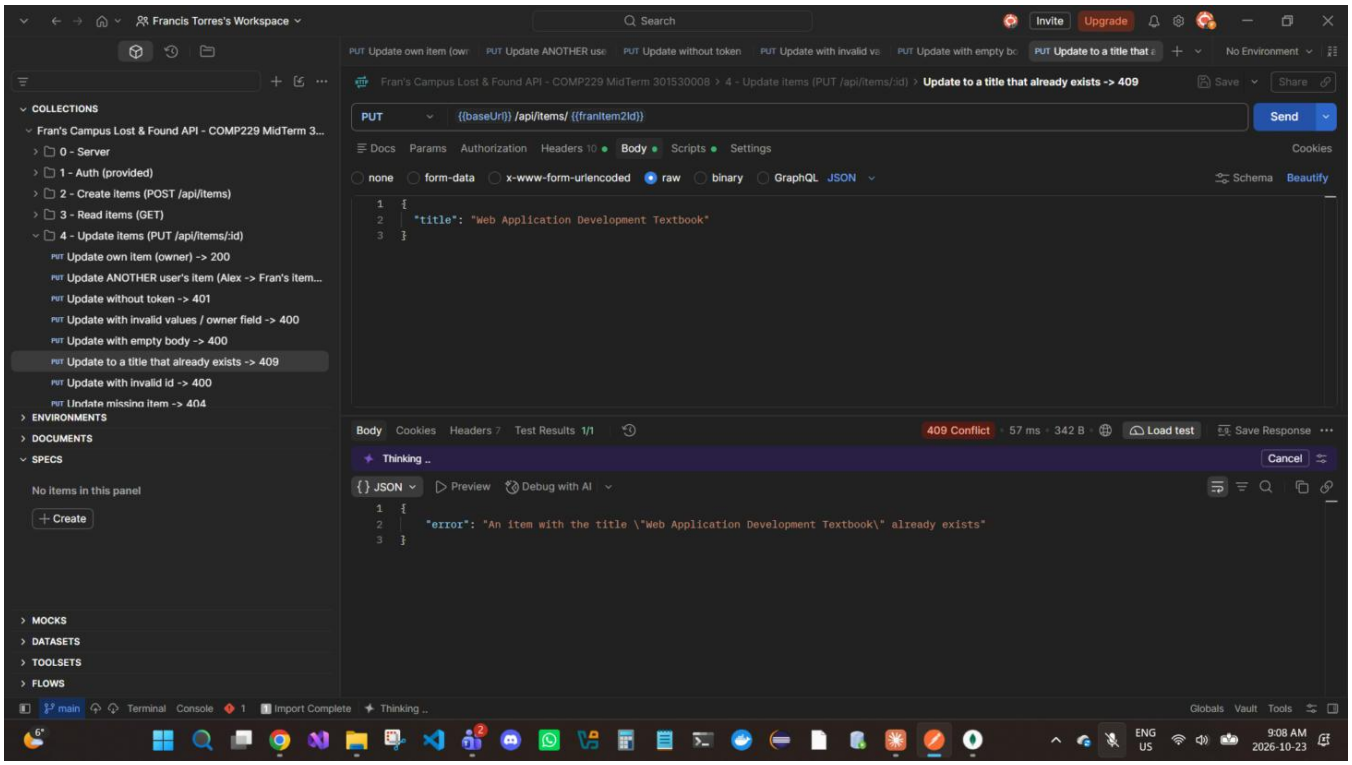


Figure 39 — PUT `/api/items/{{franItem2Id}}` — Rename to a title already used by another item → **409 Conflict**

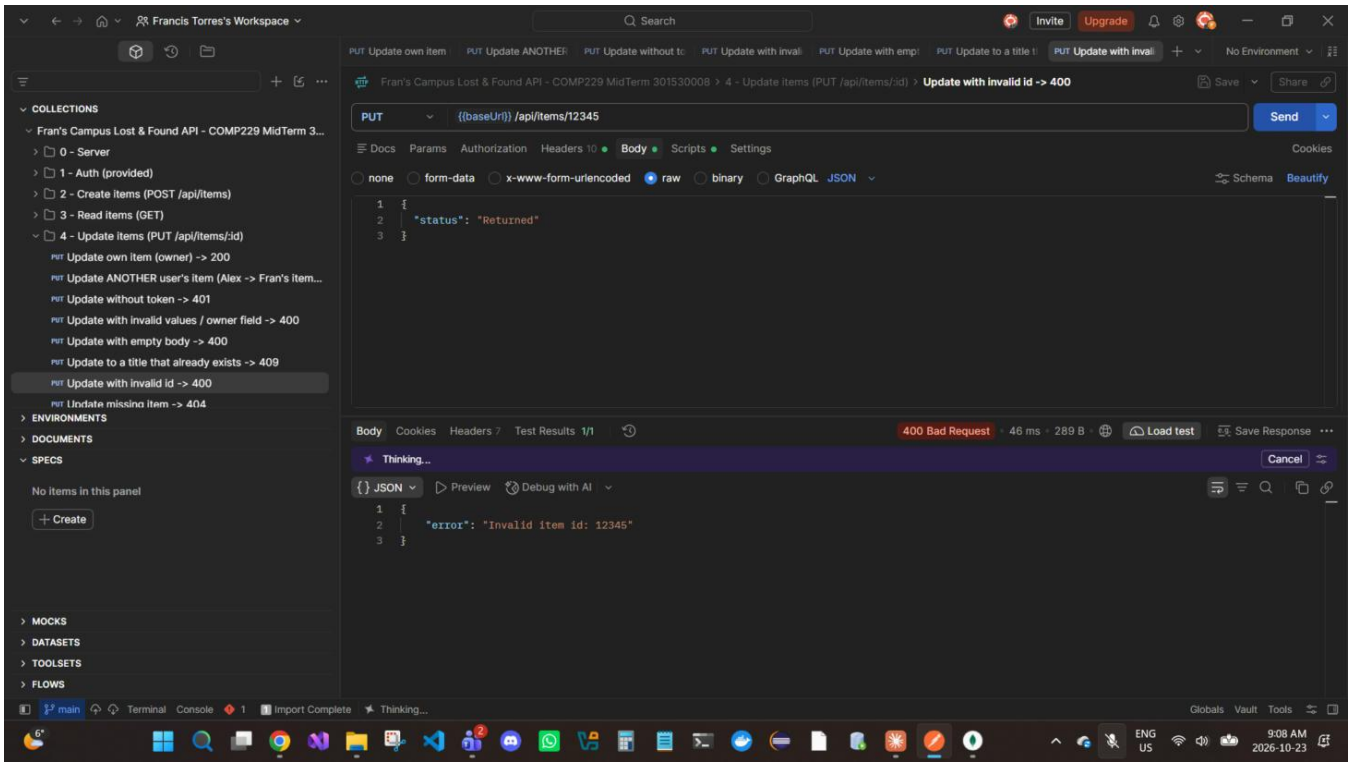


Figure 40 — PUT `/api/items/12345` — Malformed ObjectId → **400 Bad Request**

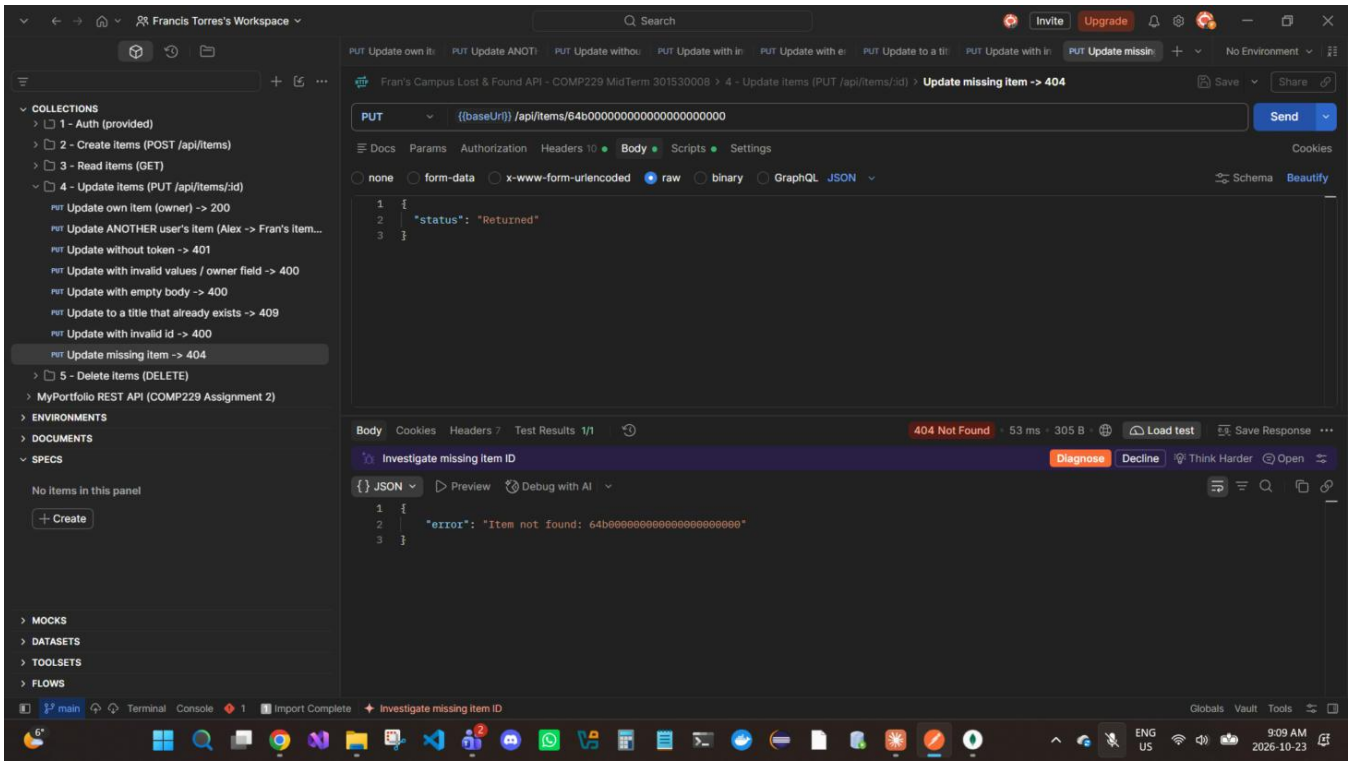


Figure 41 — PUT `/api/items/64b000000000000000000000` — Item does not exist → 404 Not Found

MongoDB evidence — Update items

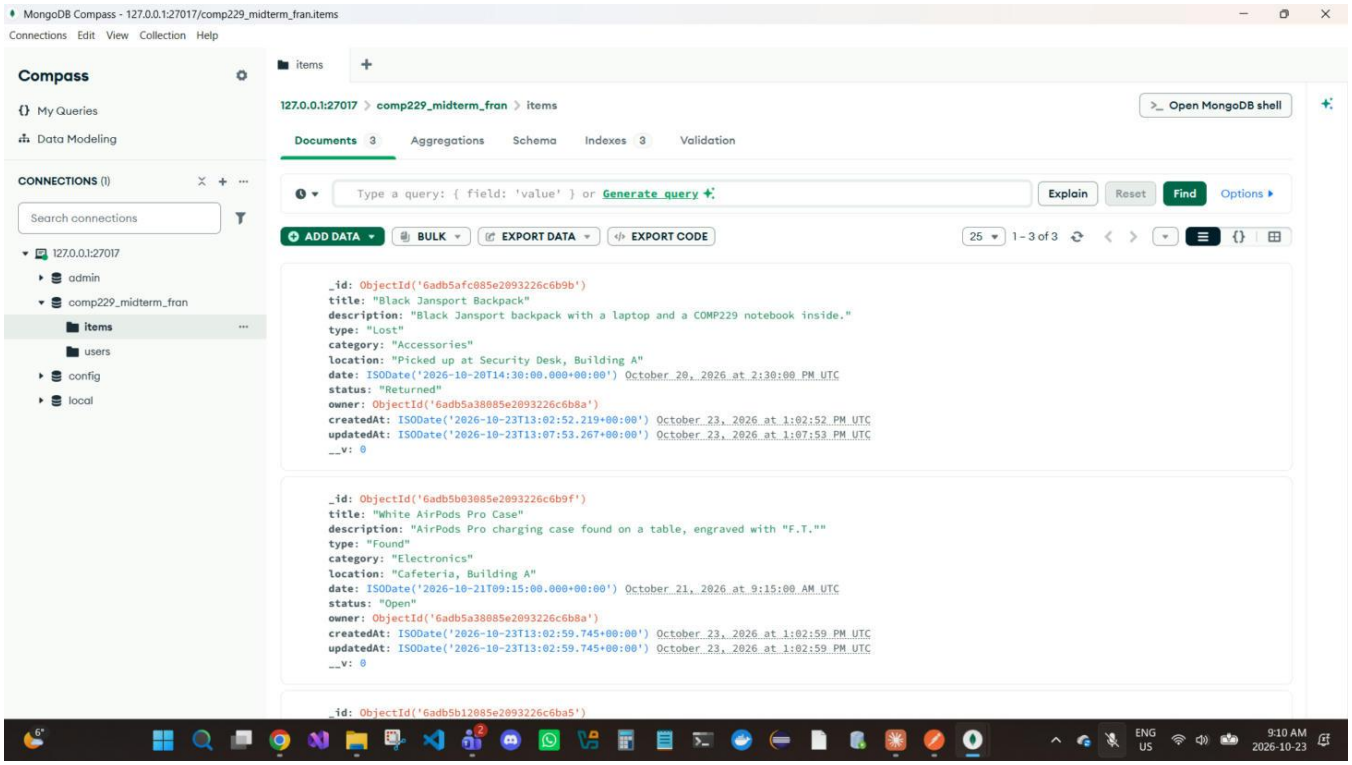


Figure 42 — MongoDB Compass after the updates: the backpack now has status "Returned", location "Picked up at Security Desk, Building A" and a newer updatedAt (1:07:53 PM vs createdAt 1:02:52 PM). Its title is still "Black Jansport Backpack", proving Alex's 403 update never reached the database.

6. Delete items — DELETE (Bearer, owner only)

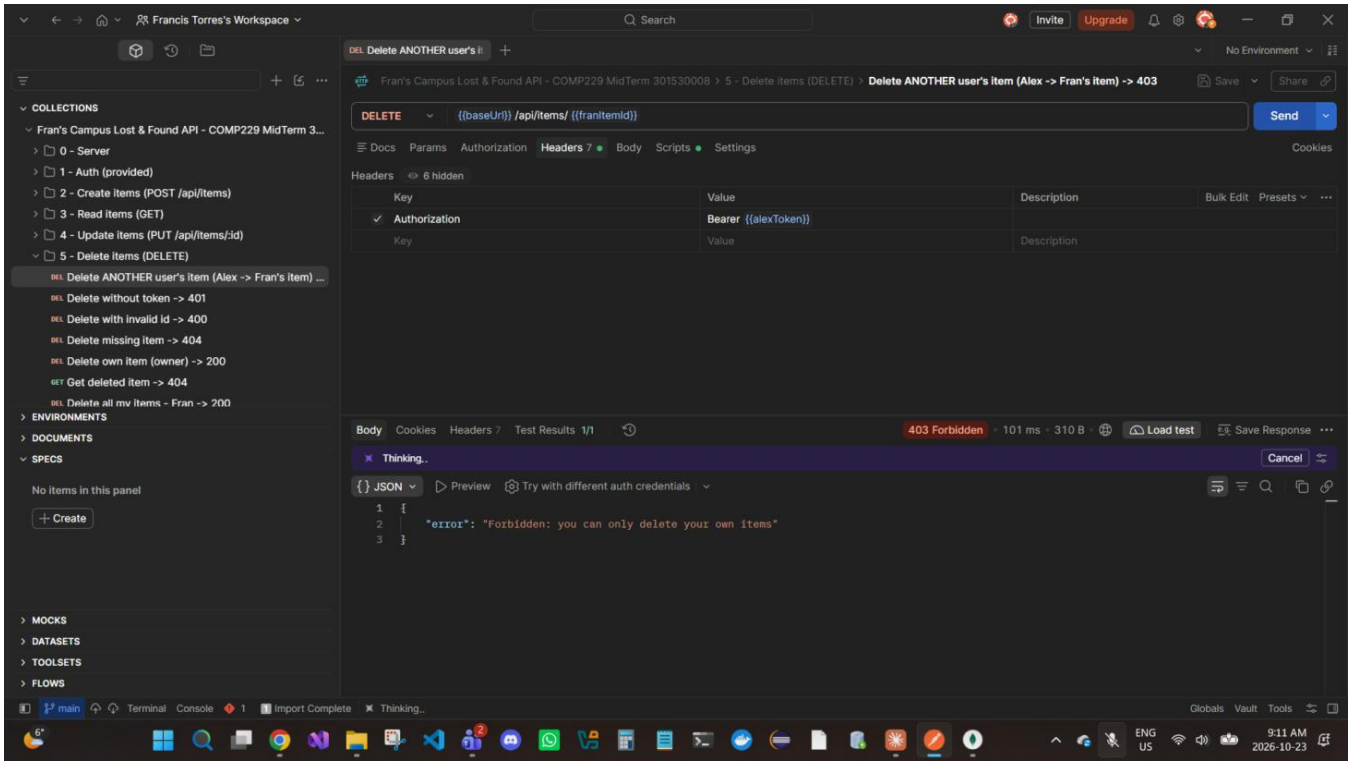


Figure 43 — `DELETE /api/items/{{franItemId}}` — OWNERSHIP: Alex tries to delete Fran's item (Bearer {{alexToken}}) — rejected → **403 Forbidden**

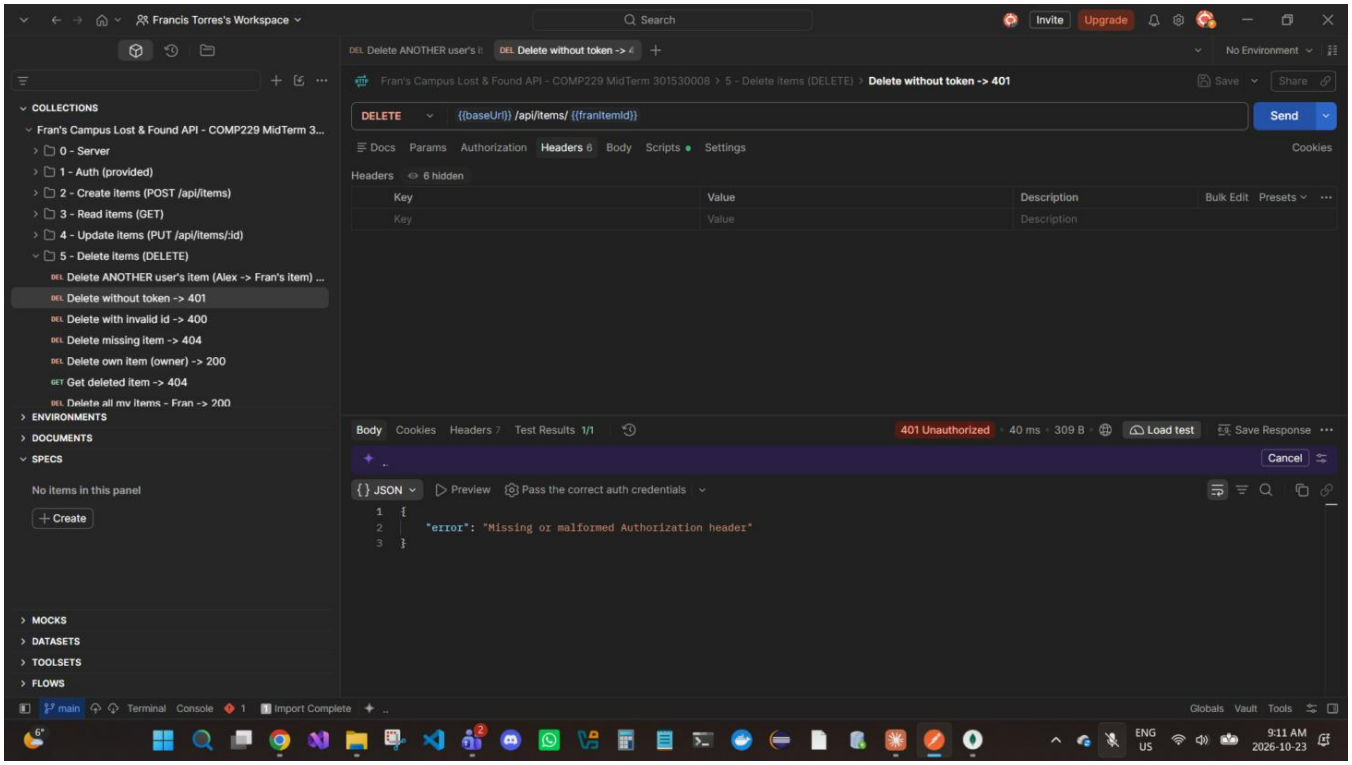


Figure 44 — `DELETE /api/items/{{franItemId}}` — Delete without a token → **401 Unauthorized**

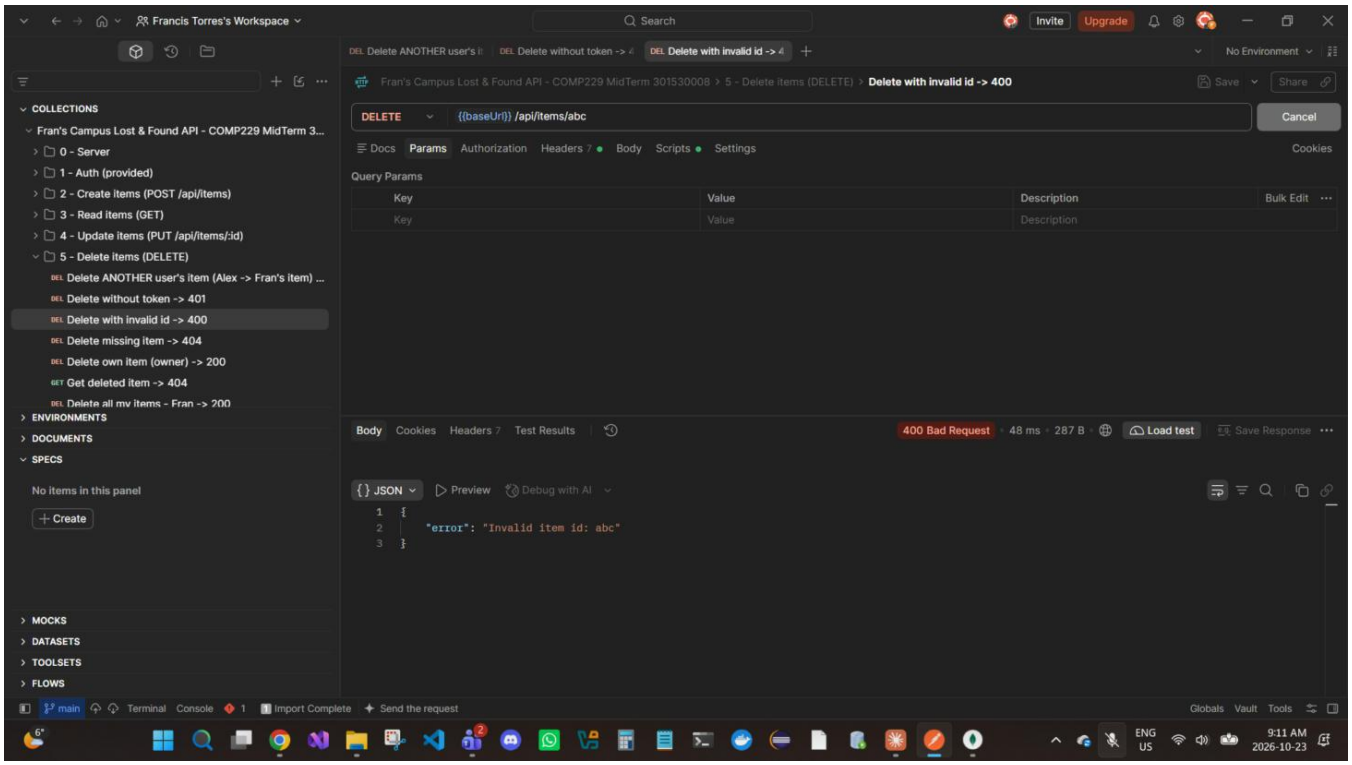


Figure 45 — `DELETE /api/items/abc` — Malformed ObjectId → 400 Bad Request

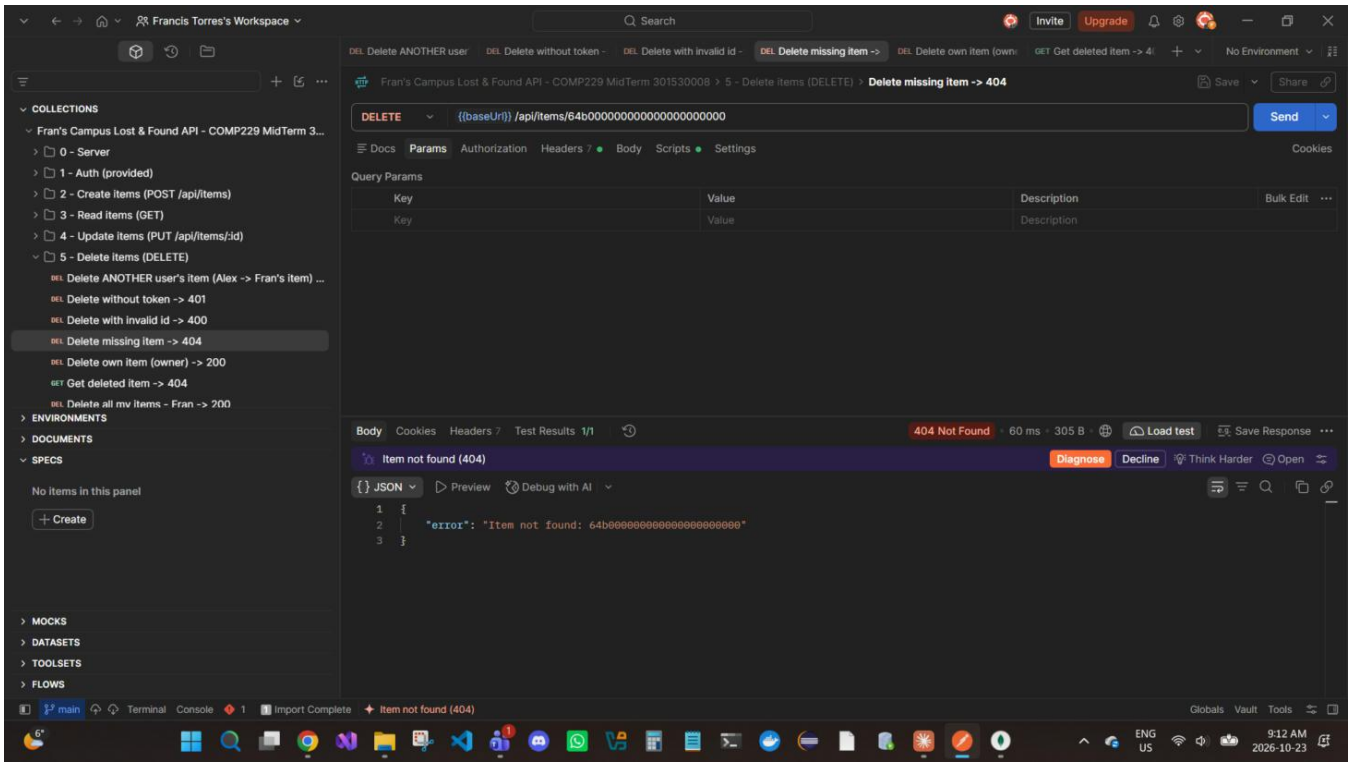


Figure 46 — `DELETE /api/items/64b000000000000000000000` — Item does not exist → 404 Not Found

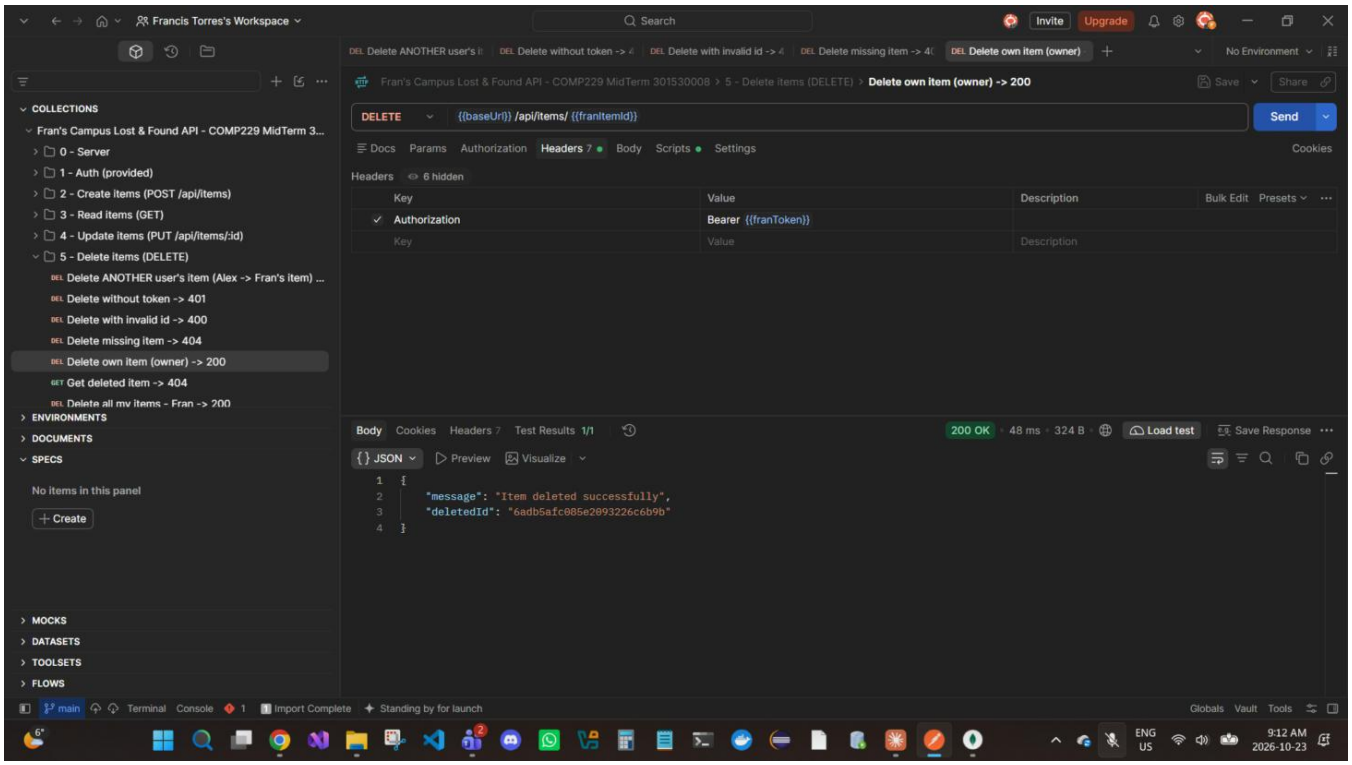


Figure 47 — `DELETE /api/items/{{franItemId}}` — Owner (Fran) deletes the backpack → **200 OK**

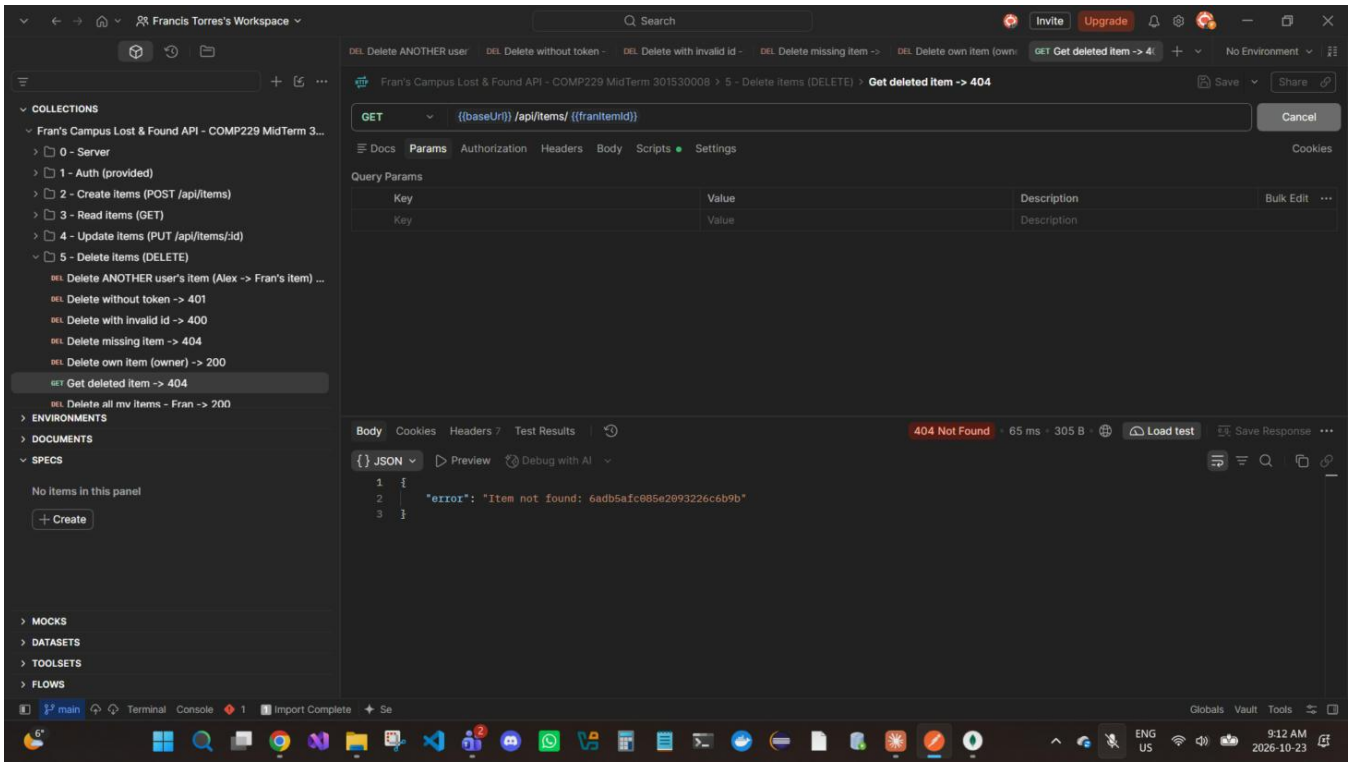


Figure 48 — `GET /api/items/{{franItemId}}` — The deleted item is gone → **404 Not Found**

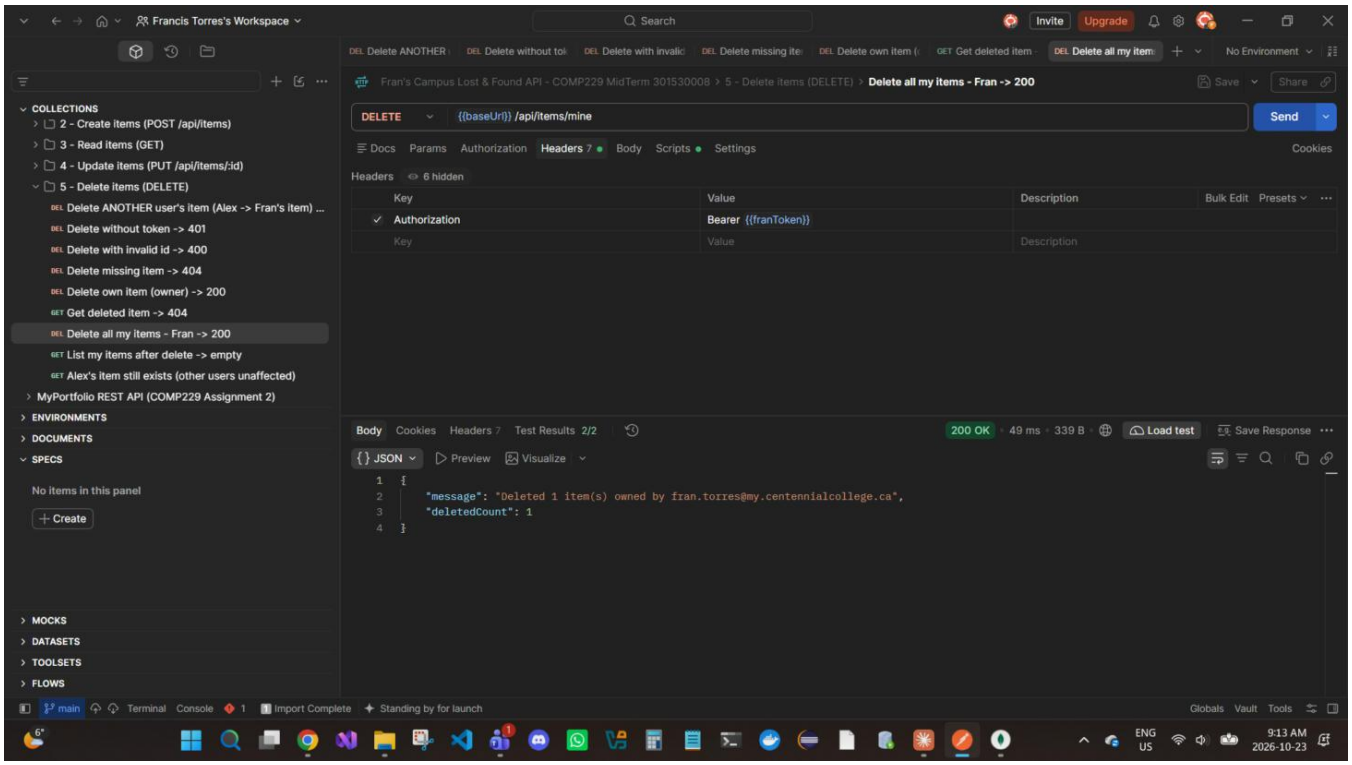


Figure 49 — `DELETE /api/items/mine` — Delete all of Fran's items; deletedCount = 1 (the AirPods) → **200 OK**

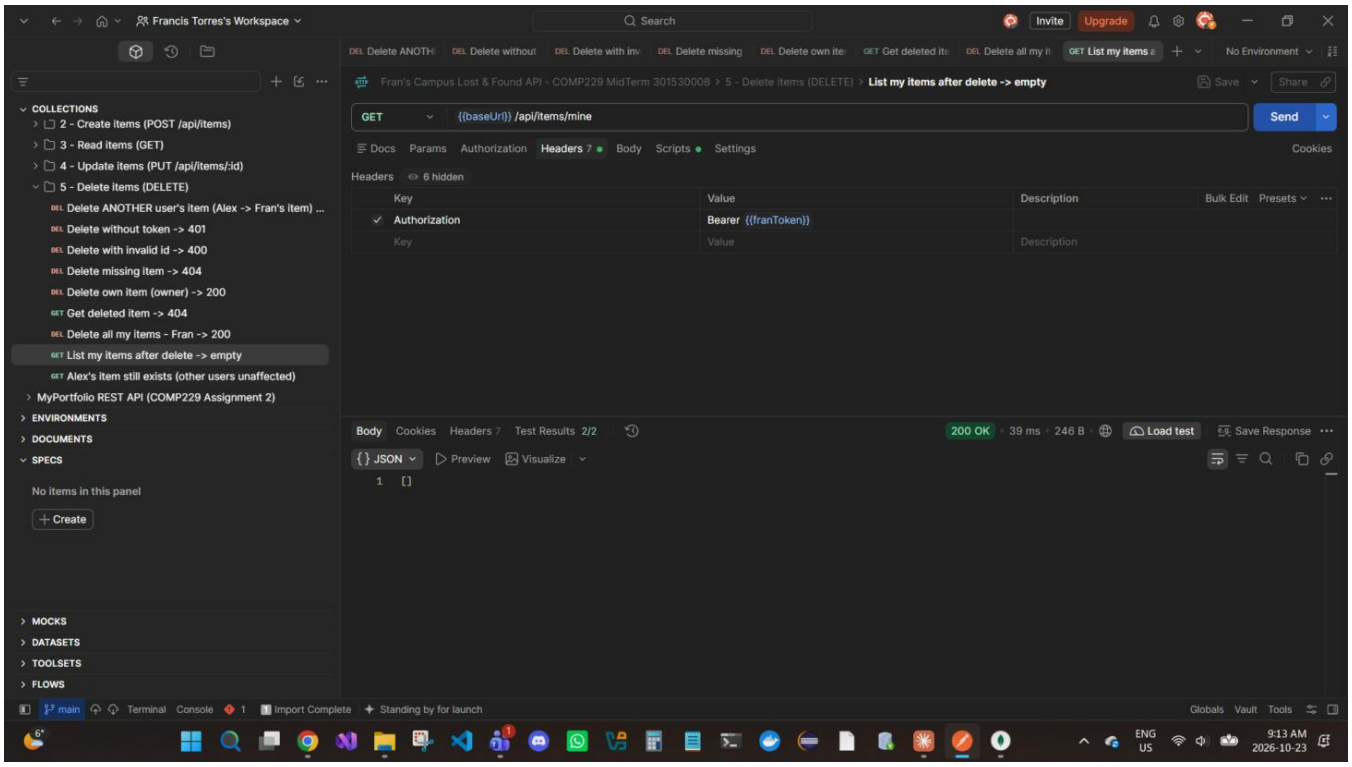


Figure 50 — `GET /api/items/mine` — Fran now has no items: [] → **200 OK**

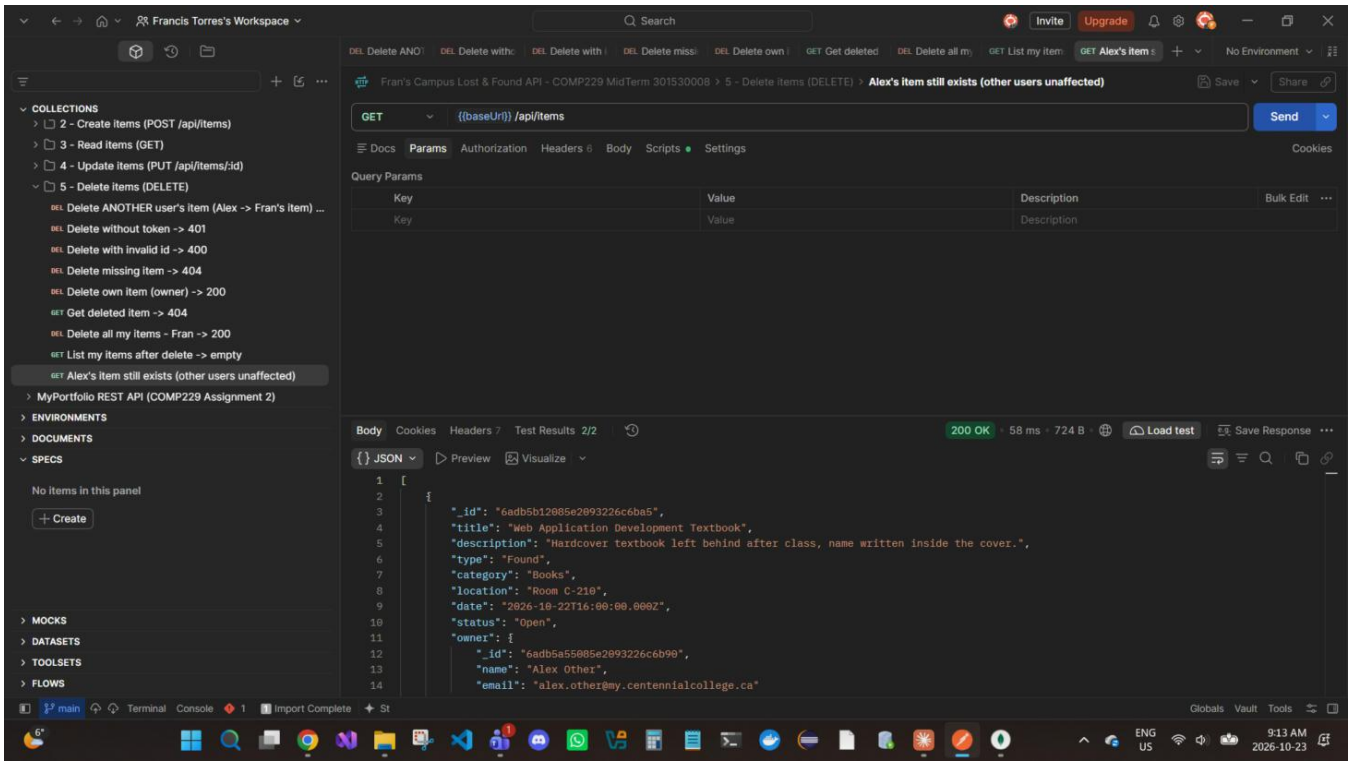


Figure 51 — `GET /api/items` — Alex's item is unaffected by Fran's DELETE /mine → 200 OK

MongoDB evidence — Delete items

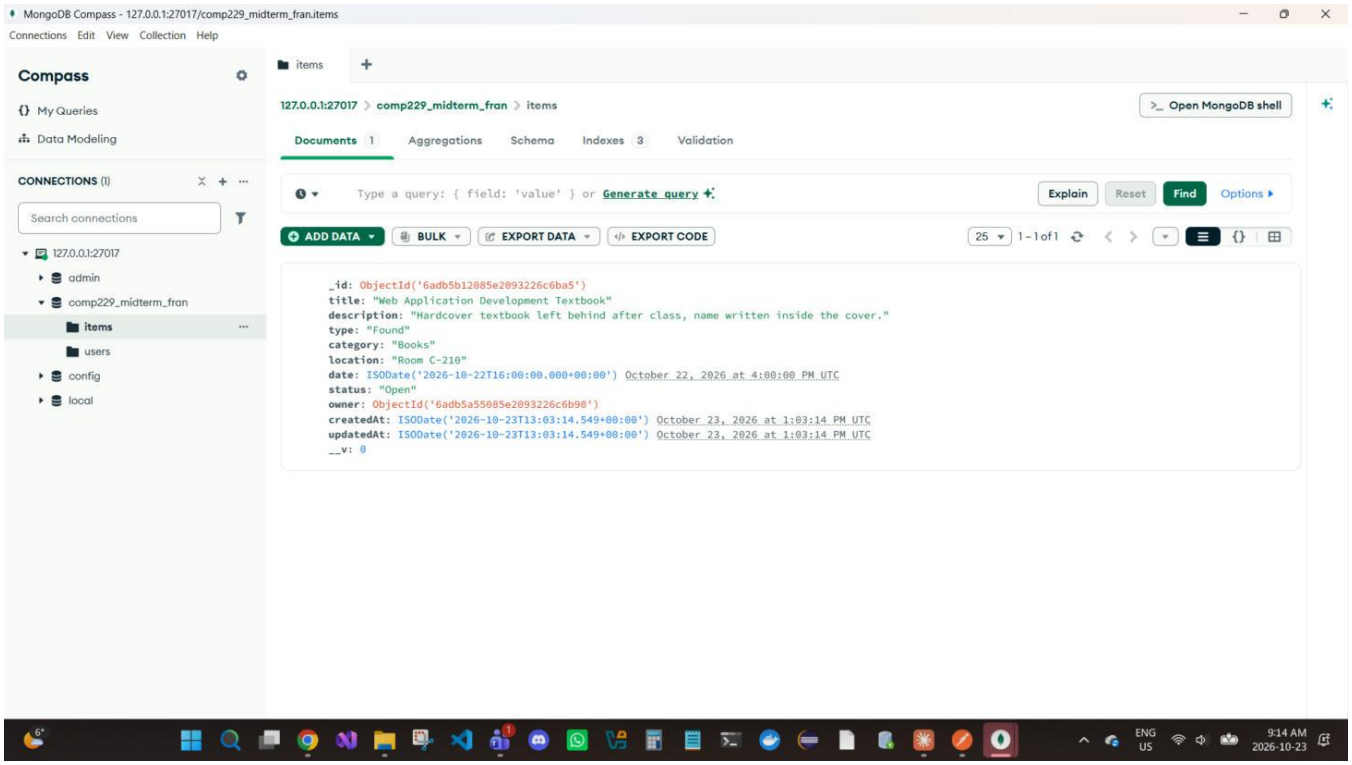


Figure 52 — MongoDB Compass after the deletes: items shows 1 of 1 — only Alex's textbook remains. Fran's two items were removed from the database.